Digital ASIC Design: RTL to GDS-II

(DSE – 5)

Practical File

Submitted By

SACHIN SHARMA
Roll No.: 23294917065 (Batch: ECE A - A1(G2)
B. Tech Electronics and Communication Engineering
(Semester VII)

To

Dr. Khushwant Sehra
(Assistant Professor)
Department of Electronics and Communication Engineering



FACULTY OF TECHNOLOGY
UNIVERSITY OF DELHI
NEW DELHI – 110007
(2025 – 2026)

Record of Experiments

S. No.	Experiment Title	Page	Remarks
1	Implementation of Basic Logic Gates	1	
2	Implementation of a 4-to-1 Multiplexer	4	
3	Implementation of a Single-Bit Full Adder	7	
4	Implementation of a Ripple Carry Adder	10	
5	Basic Logic Gates Using Structural Modeling	14	
6	4-to-1 Multiplexer Using Structural Modeling	17	
7	Implementation of a Full Subtractor	21	
8	Basic Logic Gates Using Behavioral Modeling	25	
9	Mixed Dataflow and Procedural Modeling	28	
10	2-to-1 Multiplexer Using Behavioral Modeling	32	
11	4-to-1 Multiplexer Using Behavioral Modeling	36	
12	3-to-8 Decoder with Enable	39	
13	Multiplication and Division Using Shift Operations	42	
14	Implementation of a Decoder Using Shift Operations	44	
15	Basic Logic Gates Using OpenLane	51	
16	NAND Gate and Clocked D Latch Using OpenLane	55	
17	Clocked D Latch Using Blocking and Non-Blocking Assignments	59	
18	Hamming Weight Calculation	63	
19	Leading-Zero Detector	67	
20	4-bit Up Counter with Enable and Reset	71	
21	Sequence Detector for 101 with Overlapping Detection	76	
22	Implementation of a Vending Machine Controller	81	
23	Rising-Edge Detector with Digital Switch	86	
24	Sensor-Based Traffic Signal Controller	90	

Experiment 1
Implementation of Basic Logic Gates

1.1 Aim

To implement and verify the basic logic gates AND, OR, NOT, NAND, NOR, XOR and XNOR using Verilog HDL (dataflow modeling).

1.2 Apparatus

EDA Playground (Icarus Verilog simulator, Yosys for RTL schematic and EPWave for waveform viewing).

1.3 Theory

Basic logic gates are the building blocks of every digital circuit. In dataflow modeling each gate is described using a continuous assignment (`assign`) with the corresponding bitwise operator: `&` (AND), `|` (OR), `~` (NOT), `^` (XOR). NAND, NOR and XNOR are obtained by inverting the output of AND, OR and XOR respectively.

Table 1.1: Truth Table of Basic Gates

A	B	AND	OR	NOT A	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	1	0	1
0	1	0	1	1	1	0	1	0
1	0	0	1	0	1	0	1	0
1	1	1	1	0	0	0	0	1

1.4 Verilog Code

Listing 1.1: Design: Basic Logic Gates

```
`timescale 1ns/1ps
module gates(
  input a,b,
  output y_or,y_nor,y_and,y_nand,y_xor,y_xnor,y_nota,y_notb
);
  assign y_or=a|b;
  assign y_nor=~(a|b);
  assign y_and=a & b;
  assign y_nand=~(a & b);
  assign y_xor=a^b;
  assign y_xnor=~(a^b);
  assign y_nota=~a;
  assign y_notb=~b;
endmodule
```

Listing 1.2: Testbench: Basic Logic Gates

```
`timescale 1ns/1ps

module gates_tb(

);
  reg a,b;
  wire y_or,y_nor,y_and,y_nand,y_xor,y_xnor,y_nota,y_notb;

  gates uut(a,b,y_or,y_nor,y_and,y_nand,y_xor,y_xnor,y_nota,y_notb);

  initial begin

    $dumpfile("gates.vcd");
    $dumpvars(1,gates_tb);

    a=0;
    b=0;
    #10;
    a=0;
    b=1;
    #10;
    a=1;
    b=0;
    #10;
    a=1;
    b=1;
    #10;
    $finish;
  end
endmodule
```

1.5 RTL Schematic

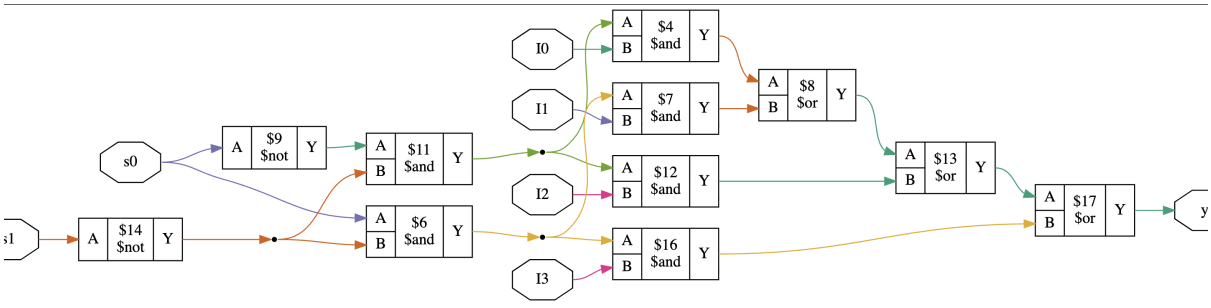


Figure 1.1: RTL schematic of basic logic gates

1.6 Simulation Output

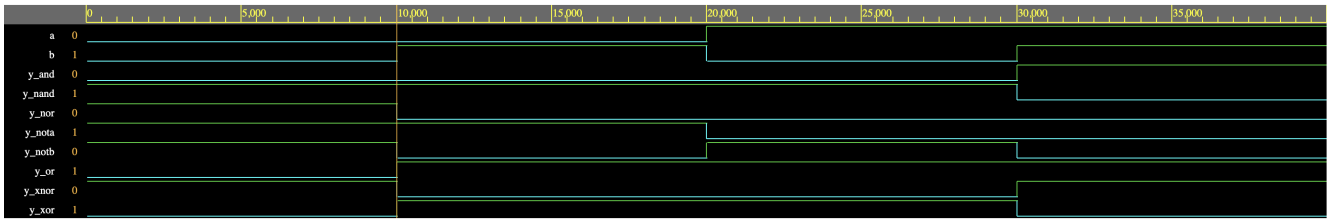


Figure 1.2: Simulation waveform of basic logic gates

1.7 Conclusion

All seven basic gates were implemented and their outputs match the truth table for every input combination.

Experiment 2
Implementation of a 4-to-1 Multiplexer

2.1 Aim

To design and implement a 4-to-1 multiplexer using select lines in Verilog HDL.

2.2 Apparatus

EDA Playground (Icarus Verilog, Yosys, EPWave).

2.3 Theory

A multiplexer selects one of several data inputs and forwards it to a single output, based on the select lines. A 4-to-1 multiplexer has four data inputs I_0–I_3, two select lines S_1S_0 and one output Y. The Boolean expression is

$$Y = \overline{S_1}\,\overline{S_0}\,I_0 + \overline{S_1}\,S_0\,I_1 + S_1\,\overline{S_0}\,I_2 + S_1\,S_0\,I_3$$

Table 2.1: Function Table of 4-to-1 Multiplexer

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

2.4 Verilog Code

Listing 2.1: Design: 4-to-1 Multiplexer (Dataflow)

```
1
2  `timescale 1ns/1ps
3
4  module mux(
5    input I0,I1,I2,I3,s0,s1,
6    output y
7  );
8    assign y = ((~s1)&(~s0)&I0) |((~s1)&( s0)&I1) |
9                (( s1)(~s0)&I2) |(( s1)&( s0)&I3);
10
11 endmodule
```

Listing 2.2: Testbench: 4-to-1 Multiplexer

```
`timescale 1ns/1ps

module mux_tb;

reg I0, I1, I2, I3, s0, s1;
wire y;

mux uut(I0, I1, I2, I3, s0, s1, y);

initial begin
$dumpfile("mux.vcd");
$dumpvars(1, mux_tb);

$monitor("Time=%0t | I3=%b | I2=%b | I1=%b | I0=%b |
          s1=%b | s0=%b | y=%b",
                          $time, I3, I2, I1, I0, s1, s0, y);


I0 = 1;
I1 = 0;
I2 = 0;
I3 = 1;

s1 = 0;
s0 = 0;
#10;

s1 = 0;
s0 = 1;
#10;
s1 = 1;
s0 = 0;
#10;

s1 = 1;
s0 = 1;
#10;

$finish;

end

endmodule
```

2.5 RTL Schematic

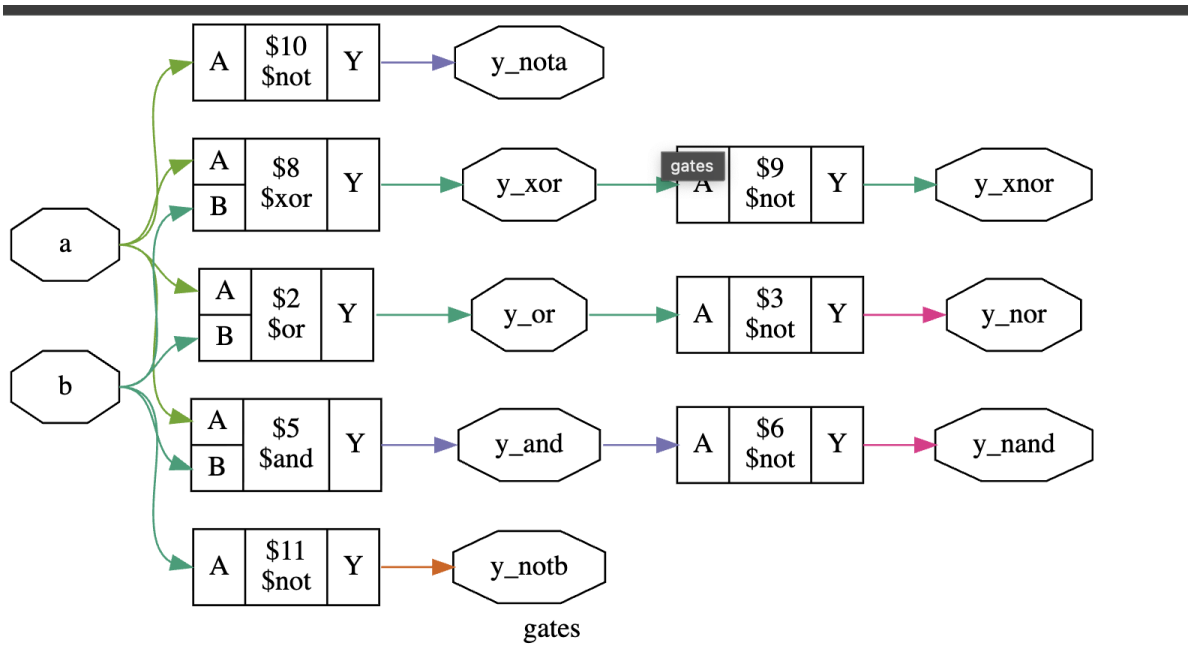


Figure 2.1: RTL schematic of 4-to-1 multiplexer

2.6 Simulation Output

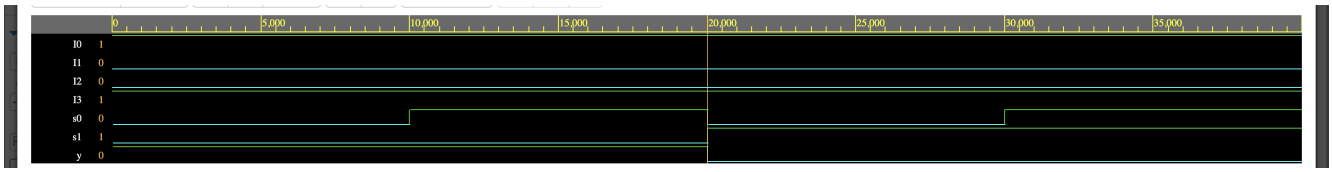


Figure 2.2: Simulation waveform of 4-to-1 multiplexer

2.7 Conclusion

The 4-to-1 multiplexer was implemented using dataflow modeling. The testbench verified all 64 combinations of data and select inputs, and the output matched the expected function table in every case.

Experiment 3
Implementation of a Single-Bit Full Adder

3.1 Aim

To design and implement a single-bit full adder with inputs A, B and Cin, producing Sum and Carry outputs.

3.2 Apparatus

EDA Playground (Icarus Verilog, Yosys, EPWave).

3.3 Theory

A full adder adds three single-bit inputs (A, B and the carry-in Cin) and produces a sum bit and a carry-out bit:

$$\text{Sum} = A \oplus B \oplus C_{in}, \qquad C_{out} = AB + (A \oplus B)C_{in}$$

Table 3.1: Truth Table of Full Adder

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

3.4 Verilog Code

Listing 3.1: Design: Full Adder

```
module full_adder(
    input A,
    input B,
    input Cin,
    output Sum,
    output Cout
);

assign Sum = A ^ B ^ Cin;
```

```
assign Cout = (A & B) | (B & Cin) | (A & Cin);

endmodule
```

Listing 3.2: Testbench: Full Adder

```
`timescale 1ns/1ps

module full_adder_tb;

reg A, B, Cin;
wire Sum, Cout;

full_adder uut(
    .A(A),
    .B(B),
    .Cin(Cin),
    .Sum(Sum),
    .Cout(Cout)
);

integer i;

initial begin
    $dumpfile("full_adder.vcd");
    $dumpvars(0, full_adder_tb);

    $monitor("Time=%0t A=%b B=%b Cin=%b | Sum=%b Cout=%b",
             $time, A, B, Cin, Sum, Cout);

    for (i = 0; i < 8; i = i + 1) begin
        {A, B, Cin} = i[2:0];
        #10;
    end

    $finish;
end

endmodule
```

3.5 RTL Schematic

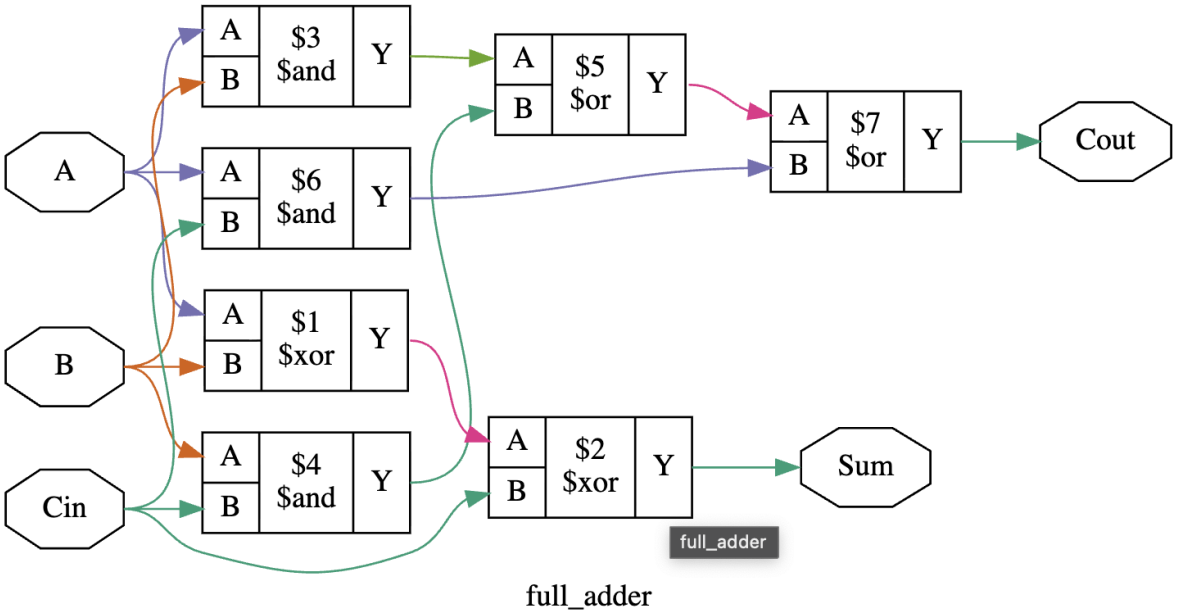


Figure 3.1: RTL schematic of the full adder

3.6 Simulation Output

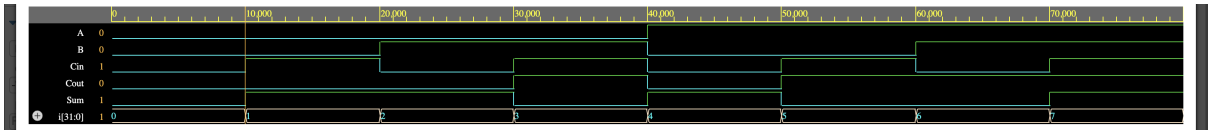


Figure 3.2: Simulation waveform of the full adder

3.7 Conclusion

The full adder produces the correct Sum and Cout for all eight input combinations, matching the truth table.

Experiment 4
Implementation of a Ripple Carry Adder

4.1 Aim

To design and implement a 4-bit ripple carry adder by cascading single-bit full adders.

4.2 Apparatus

EDA Playground (Icarus Verilog, Yosys, EPWave).

4.3 Theory

A ripple carry adder is formed by connecting the carry-out of each full adder to the carry-in of the next stage. The carry "ripples" from the least significant bit to the most significant bit, so the total delay grows linearly with the number of bits.

A single-bit full adder adds three bits, A, B and the carry-in C_{in}, and produces a sum bit and a carry-out bit:

$$\text{Sum} = A \oplus B \oplus C_{in}, \qquad C_{out} = AB + BC_{in} + AC_{in}$$

The carry-out is 1 whenever at least two of the three inputs are 1 (majority function). For a 4-bit adder, four full adders are cascaded. The carry-in of the first stage is the external input C_{in}, and the carry-out of stage i becomes the carry-in of stage $i+1$:

$$C_1 = A_0B_0 + B_0C_{in} + A_0C_{in}, \quad C_2 = A_1B_1 + B_1C_1 + A_1C_1, \quad \ldots$$

The final carry C_4 is the carry-out C_{out} of the adder. The result is a 5-bit number $\{C_{out}, S_3S_2S_1S_0\}$.

Table 4.1: Truth Table of Single-Bit Full Adder

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

4.4 Verilog Code

Listing 4.1: Design: 4-bit Ripple Carry Adder

```
// 1-bit Full Adder
module full_adder(
    input A,
    input B,
    input Cin,
    output Sum,
    output Cout
);

assign Sum  = A ^ B ^ Cin;
assign Cout = (A & B) | (B & Cin) | (A & Cin);

endmodule


// 4-bit Ripple Carry Adder
module ripple_adder_4bit(
    input  [3:0] A,
    input  [3:0] B,
    input        Cin,
    output [3:0] Sum,
    output       Cout
);

wire C1, C2, C3;

full_adder FA0(A[0], B[0], Cin, Sum[0], C1);
full_adder FA1(A[1], B[1], C1,  Sum[1], C2);
full_adder FA2(A[2], B[2], C2,  Sum[2], C3);
full_adder FA3(A[3], B[3], C3,  Sum[3], Cout);

endmodule
```

Listing 4.2: Testbench: 4-bit Ripple Carry Adder

```
`timescale 1ns/1ps

module ripple_adder_4bit_tb;

reg  [3:0] A, B;
reg        Cin;
wire [3:0] Sum;
wire       Cout;

ripple_adder_4bit uut (A,B,C,Sum,Cout);

integer i, j, k;

initial begin
    $dumpfile("ripple_adder_4bit.vcd");
    $dumpvars(0, ripple_adder_4bit_tb);

    $monitor("Time=%0t A=%b B=%b Cin=%b | Sum=%b Cout=%b",
             $time, A, B, Cin, Sum, Cout);
```

```
20
21      for (i = 0; i < 16; i = i + 1) begin
22          for (j = 0; j < 16; j = j + 1) begin
23              for (k = 0; k < 2; k = k + 1) begin
24
25                  A = i;
26                  B = j;
27                  Cin = k;
28
29                  #10;
30
31              end
32          end
33      end
34
35      $finish;
36  end
37
38  endmodule
```

4.5 RTL Schematic

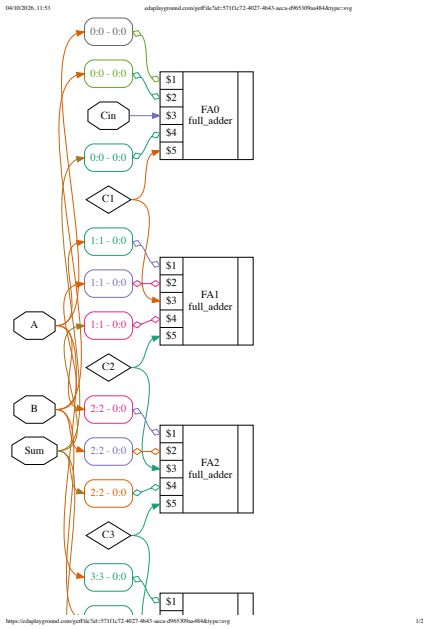


Figure 4.1: RTL schematic of 4-bit ripple carry adder

4.6 Simulation Output

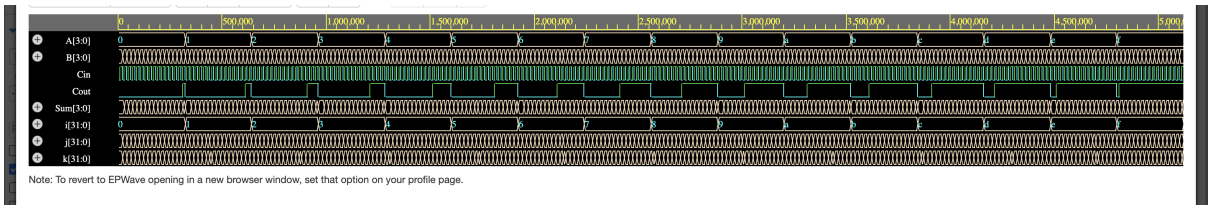


Figure 4.2: Simulation waveform of 4-bit ripple carry adder

4.7 Observation Table

The testbench covers all $16 \times 16 \times 2 = 512$ input combinations, each applied for 10 ns. Some representative cases are listed below.

Table 4.2: Observed Output of 4-bit Ripple Carry Adder (Selected Cases)

A	B	C_{in}	Sum	C_{out}	Decimal Result
0011	0101	0	1000	0	$3 + 5 + 0 = 8$
0111	1000	1	0000	1	$7 + 8 + 1 = 16$
1111	0001	0	0000	1	$15 + 1 + 0 = 16$
1001	1001	1	0011	1	$9 + 9 + 1 = 19$
1111	1111	1	1111	1	$15 + 15 + 1 = 31$

4.8 Result

The 4-bit ripple carry adder was designed and simulated successfully. The testbench applied all 512 combinations of A, B and C_{in}, and the simulated Sum and Cout matched the expected arithmetic result.

4.9 Conclusion

A 4-bit ripple carry adder was built by cascading four single-bit full adders. The sum and carry-out matched the expected result in the observed cases, including those where the carry propagates through all four stages (for example, $15 + 1$ and $15 + 15 + 1$). The experiment also showed that the delay of this adder grows with the number of bits because each stage must wait for the carry of the previous stage.

Experiment 5
Basic Logic Gates Using Structural Modeling

5.1 Aim

To implement basic logic gates using structural modeling by describing the circuit in terms of gate-level components.

5.2 Apparatus

EDA Playground (Icarus Verilog, Yosys, EPWave).

5.3 Theory

In structural modeling the circuit is described by instantiating built-in gate primitives (`and`, `or`, `not`, `nand`, `nor`, `xor`, `xnor`) and connecting them with wires. The first terminal of each primitive is the output, followed by the inputs.

Structural modeling is closest to the actual hardware: each primitive corresponds to a physical gate and each wire to a connection between gates. The general syntax of a gate instantiation is

```
gate_type instance_name (output, input1, input2, ...);
```

For multi-input gates such as `and`, `or`, `nand`, `nor`, `xor` and `xnor` there is one output and any number of inputs, while `not` (and `buf`) has one input and one output. The instance name is optional but is useful for identifying gates in the schematic and the waveform.

5.4 Verilog Code

Listing 5.1: Design: Basic Gates (Structural)

```
`timescale 1ns/1ps

module gates_struct(
    input a, b,
    output y1, y2, y3, y4, y5, y6, y7
);

and  g1(y1, a, b);   // AND
or   g2(y2, a, b);   // OR
not  g3(y3, a);      // NOT
nand g4(y4, a, b);   // NAND
nor  g5(y5, a, b);   // NOR
xor  g6(y6, a, b);   // XOR
xnor g7(y7, a, b);   // XNOR

endmodule
```

Listing 5.2: Testbench: Basic Gates (Structural)

```
`timescale 1ns/1ps

module gates_struct_tb;

reg a, b;
wire y1, y2, y3, y4, y5, y6, y7;

gates_struct uut(a, b, y1, y2, y3, y4, y5, y6, y7);

initial begin
    $dumpfile("gates_struct.vcd");
    $dumpvars(0, gates_struct_tb);

    $monitor("Time=%0t | a=%b b=%b | AND=%b OR=%b NOTa=%b NAND=%b NOR=%
        b XOR=%b XNOR=%b",
              $time, a, b, y1, y2, y3, y4, y5, y6, y7);

    a = 0; b = 0; #10;
    a = 0; b = 1; #10;
    a = 1; b = 0; #10;
    a = 1; b = 1; #10;

    $finish;
end

endmodule
```

5.5 RTL Schematic

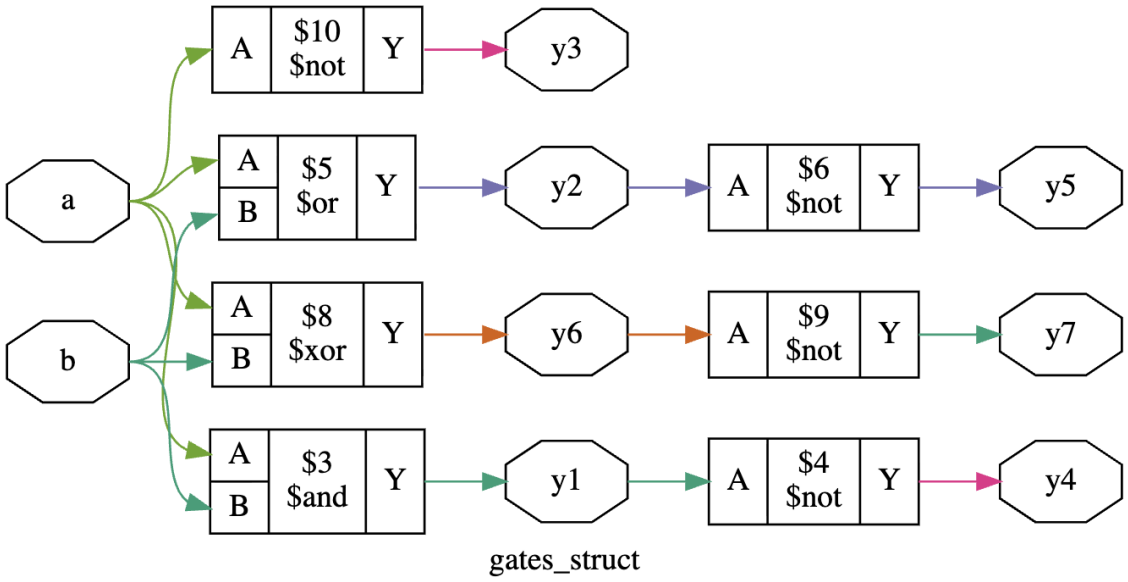


Figure 5.1: RTL schematic of basic gates (structural modeling)

5.6 Simulation Output

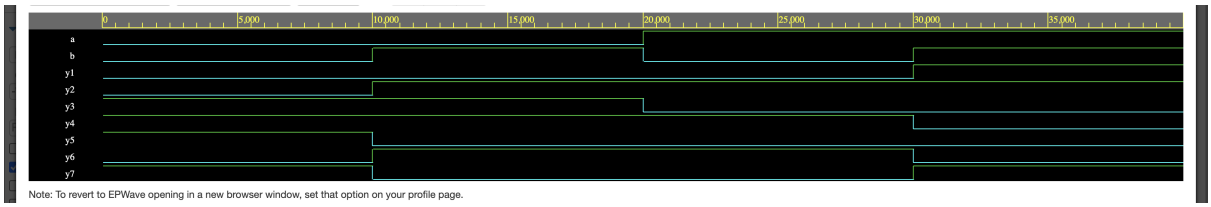


Figure 5.2: Simulation waveform of basic gates (structural modeling)

5.7 Observation Table

Table 5.1: Observed Outputs of Structural Gate Model

a	b	y1 (AND)	y2 (OR)	y3 (NOT a)	y4 (NAND)	y5 (NOR)	y6 (XOR)	y7 (XNOR)
0	0	0	0	1	1	1	0	1
0	1	0	1	1	1	0	1	0
1	0	0	1	0	1	0	1	0
1	1	1	1	0	0	0	0	1

5.8 Result

All seven basic gates were implemented using gate primitives in structural modeling, and the simulated outputs matched the truth table for every input combination.

5.9 Conclusion

Structural modeling describes a circuit as an interconnection of gate primitives. The outputs `y1` to `y7` matched the AND, OR, NOT, NAND, NOR, XOR and XNOR truth tables, showing that the structural description behaves the same as the dataflow description of Experiment 1 while being closer to the actual gate-level hardware.

Experiment 6
4-to-1 Multiplexer Using Structural Modeling

6.1 Aim

To design and implement a 4-to-1 multiplexer using structural (gate-level) modeling.

6.2 Apparatus

EDA Playground (Icarus Verilog, Yosys, EPWave).

6.3 Theory

A 4-to-1 multiplexer can be built from two inverters (for the complemented select lines), four 3-input AND gates and one 4-input OR gate. Each AND gate enables one data input for a particular combination of the select lines, and the OR gate combines the four terms into the output Y.

The Boolean expression of the multiplexer is

$$Y = \overline{S_1}\,\overline{S_0}\,I_0 + \overline{S_1}\,S_0\,I_1 + S_1\,\overline{S_0}\,I_2 + S_1\,S_0\,I_3$$

In the structural description, each product term is produced by one `and` primitive, and the sum of the four terms is produced by one `or` primitive. The inverters `not` generate $\overline{S_0}$ and $\overline{S_1}$. The internal wires `w0`–`w3` connect the outputs of the AND gates to the inputs of the OR gate.

6.4 Verilog Code

Listing 6.1: Design: 4-to-1 Multiplexer (Structural)

```
`timescale 1ns/1ps

module mux4x1(input [3:0]I, input [1:0]S, output Y);
  wire w0,w1,w2,w3;
  wire s0bar,s1bar;

  not n1(s0bar,S[0]);
  not n2(s1bar,S[1]);

  and a1(w0,S[0],S[1],I[3]);
  and a2(w1,s0bar,S[1],I[2]);
  and a3(w2,S[0],s1bar,I[1]);
  and a4(w3,s0bar,s1bar,I[0]);

  or o1(Y,w0,w1,w2,w3);

endmodule
```

Listing 6.2: Testbench: 4-to-1 Multiplexer (Structural)

```
`timescale 1ns/1ps

module mux_tb();

    reg [3:0] I;
    reg [1:0] S;
    wire Y;

    reg expected;
    integer i, errors;

    mux4x1 uut(
        .I(I),
        .S(S),
        .Y(Y)
    );

    initial begin
        $dumpfile("mux.vcd");
        $dumpvars(0, mux_tb);

        $monitor("Time=%0t S=%b I=%b | Y=%b", $time, S, I, Y);

        errors = 0;

        for (i = 0; i < 64; i = i + 1) begin
            {S, I} = i;
            expected = I[S];
            #10;
            if (Y !== expected) begin
                errors = errors + 1;
                $display("ERROR: S=%b I=%b expected=%b got=%b", S, I,
                    expected, Y);
            end
        end

        if (errors == 0) $display("PASS: all 64 cases correct");
        else             $display("FAIL: %0d errors", errors);

        $finish;
    end

endmodule
```

6.5 RTL Schematic

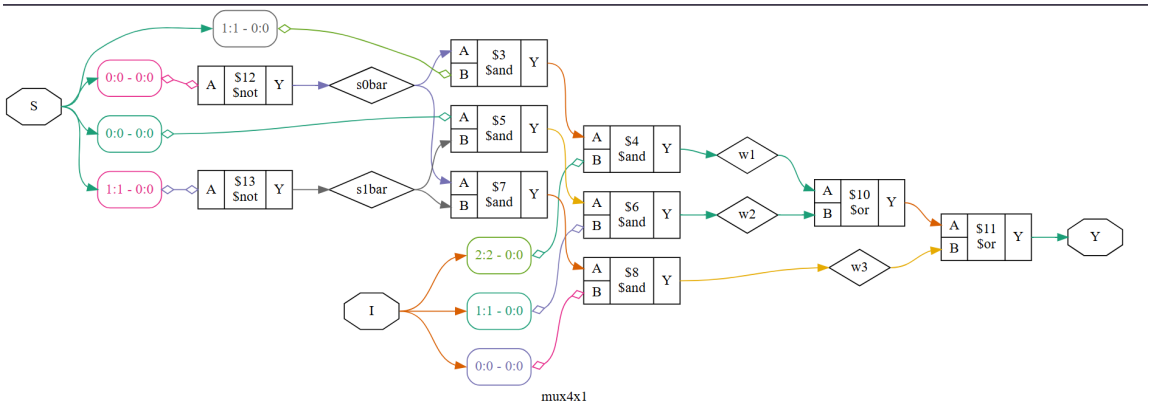


Figure 6.1: RTL schematic of the structural 4-to-1 multiplexer

6.6 Simulation Output

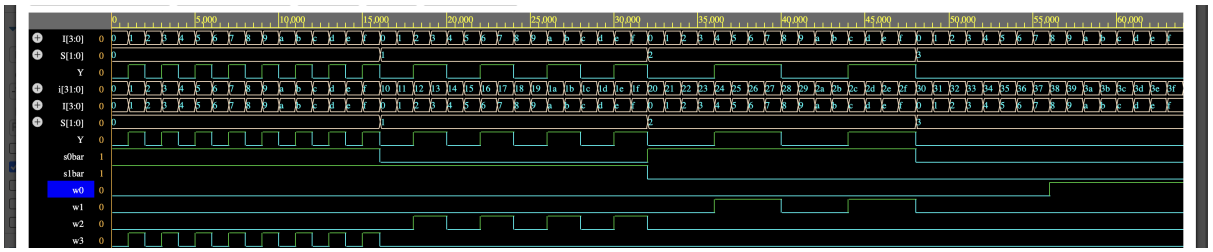


Figure 6.2: Simulation waveform of the structural 4-to-1 multiplexer

6.7 Observation Table

Table 6.1: Observed Output of Structural 4-to-1 Multiplexer (Selected Cases)

S_1S_0	$I_3I_2I_1I_0$	Selected Input	Y
00	0001	I_0	1
01	0010	I_1	1
10	0100	I_2	1
11	1000	I_3	1
11	0111	I_3	0

6.8 Result

The 4-to-1 multiplexer was implemented using gate primitives and simulated successfully. The self-checking testbench applied all 64 combinations of `S` and `I`, and the output `Y` matched the selected input in every case.

6.9 Conclusion

The 4-to-1 multiplexer was described structurally using two inverters, four 3-input AND gates and one 4-input OR gate. The output followed the data input selected by S_1S_0 in every test, which matches the function table. The structural description corresponds directly to the gate-level circuit, and the exhaustive testbench confirmed that each AND gate enables exactly one data input.

Experiment 7
Implementation of a Full Subtractor

7.1 Aim

To design and implement a full subtractor with inputs A, B and Bin, producing Difference and Borrow outputs.

7.2 Apparatus

EDA Playground (Icarus Verilog, Yosys, EPWave).

7.3 Theory

A full subtractor computes $A - B - B_{in}$ and produces a difference bit and a borrow-out bit:

$$D = A \oplus B \oplus B_{in}, \qquad B_{out} = \overline{A}B + \overline{(A \oplus B)}\, B_{in}$$

Here B_{in} is the borrow taken by the previous (less significant) stage, and B_{out} is the borrow passed to the next stage. A borrow is generated when the subtrahend ($B + B_{in}$) is larger than the minuend A. The difference output is the same XOR function as the sum of a full adder; only the borrow expression differs from the carry of the adder, because A is complemented in the first term.

The borrow expression has two terms. The term $\overline{A}B$ generates a borrow when $A = 0$ and $B = 1$. The term $\overline{(A \oplus B)}\, B_{in}$ propagates an incoming borrow when A and B are equal. In dataflow modeling both outputs are written directly as continuous assignments using bitwise operators.

Table 7.1: Truth Table of Full Subtractor

A	B	Bin	Difference	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

7.4 Verilog Code

Listing 7.1: Design: Full Subtractor

```
module full_subtractor (
    input A, B, Bin,
    output D, Bout
);

    wire term1;
    wire abar;
    wire term3;
    wire term4;
    wire term5;

    xor (term1, A, B);
    xor (D, term1, Bin);

    not (term3, term1);
    not (abar, A);

    and (term4, term3, Bin);
    and (term5, abar, B);

    or (Bout, term4, term5);

endmodule
```

Listing 7.2: Testbench: Full Subtractor

```
`timescale 1ns/1ps

module tb_full_subtractor;

    reg A, B, Bin;
    wire D, Bout;
    full_subtractor uut (
        .A(A),
        .B(B),
        .Bin(Bin),
        .D(D),
        .Bout(Bout)
    );

    initial begin

        $dumpfile("subtractor.vcd");
        $dumpvars(1, tb_full_subtractor);


        $monitor("Time=%0t | A=%b B=%b Bin=%b | D=%b Bout=%b",
                 $time, A, B, Bin, D, Bout);


        A=0; B=0; Bin=0;
        #10;

        A=0; B=0; Bin=1;
        #10;


```

```
32          A=0; B=1; Bin=0;
33          #10;
34
35
36          A=0; B=1; Bin=1;
37          #10;
38
39
40          A=1; B=0; Bin=0;
41          #10;
42
43
44          A=1; B=0; Bin=1;
45          #10;
46
47
48          A=1; B=1; Bin=0;
49          #10;
50
51          A=1; B=1; Bin=1;
52          #10;
53
54          $finish;
55
56      end
57
58  endmodule
```

7.5 RTL Schematic

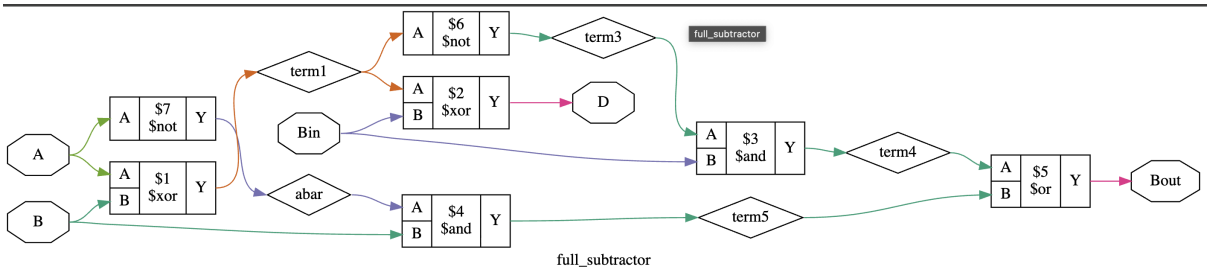


Figure 7.1: RTL schematic of full subtractor

7.6 Simulation Output

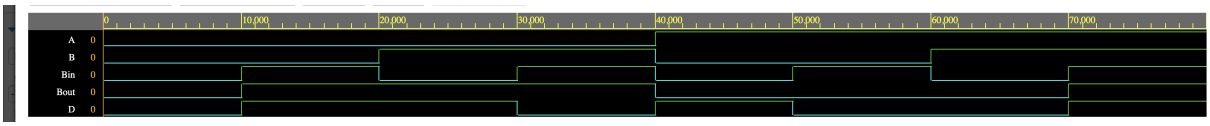


Figure 7.2: Simulation waveform of full subtractor

7.7 Observation Table

Table 7.2: Observed Output of Full Subtractor

a	b	bin	diff	borrow	Check ($a - b - b_{in}$)
0	0	0	0	0	$0 - 0 - 0 = 0$
0	0	1	1	1	$0 - 0 - 1 = -1$
0	1	0	1	1	$0 - 1 - 0 = -1$
0	1	1	0	1	$0 - 1 - 1 = -2$
1	0	0	1	0	$1 - 0 - 0 = 1$
1	0	1	0	0	$1 - 0 - 1 = 0$
1	1	0	0	0	$1 - 1 - 0 = 0$
1	1	1	1	1	$1 - 1 - 1 = -1$

7.8 Result

The full subtractor was designed and simulated successfully. The self-checking testbench applied all 8 input combinations and the outputs `diff` and `borrow` matched the expected result of $a - b - b_{in}$ in every case.

7.9 Conclusion

The full subtractor was implemented using dataflow modeling. The difference output is the XOR of the three inputs, and the borrow output is generated when $A = 0$ and $B = 1$ or when an incoming borrow is propagated through equal bits. The simulated outputs matched the truth table and the arithmetic check for all combinations.

Experiment 8
Basic Logic Gates Using Behavioral Modeling

8.1 Aim

To implement basic logic gates using behavioral modeling.

8.2 Apparatus

EDA Playground (Icarus Verilog, Yosys, EPWave).

8.3 Theory

In behavioral modeling the functionality is described using procedural blocks (`always`). Outputs assigned inside an `always` block must be declared as `reg`. The sensitivity list `@(*)` makes the block execute whenever any input changes.

Behavioral modeling describes *what* the circuit does rather than *how* it is connected. The statements inside an `always` block are executed sequentially, one after another. Here the blocking assignment operator (`=`) is used, which is the correct choice for combinational logic.

Points to Note

- Declaring an output as `reg` does not mean that a register (flip-flop) is created. Since the block is sensitive to all inputs (`@(*)`) and every output is assigned on every execution, the synthesized hardware is purely combinational.
- If an output is not assigned in some path of the block, the synthesis tool infers a latch. Here all seven outputs are assigned unconditionally, so no latch is created.
- The wildcard `@(*)` automatically includes all signals read inside the block, which avoids mistakes in a manually written sensitivity list.

Comparison of Modeling Styles

The same gates were implemented using dataflow modeling (Experiment 1: `assign`), structural modeling (Experiment 5: gate primitives) and now behavioral modeling (`always`). All three styles produce the same truth table; they differ only in the level of abstraction used to describe the circuit.

8.4 Verilog Code

Listing 8.1: Design: Basic Gates (Behavioral)

```
`timescale 1ns/1ps

module gates_beh(
    input a, b,
    output reg y1, y2, y3, y4, y5, y6, y7
);

always @(*) begin
    y1 = a & b;       // AND
    y2 = a | b;       // OR
    y3 = ~a;          // NOT
    y4 = ~(a & b);    // NAND
    y5 = ~(a | b);    // NOR
    y6 = a ^ b;       // XOR
    y7 = ~(a ^ b);    // XNOR
end

endmodule
```

Listing 8.2: Testbench: Basic Gates (Behavioral)

```
// Code your testbench here
// or browse Examples
`timescale 1ns/1ps

module gates_beh_tb;

reg a, b;
wire y1, y2, y3, y4, y5, y6, y7;

gates_beh uut(a, b, y1, y2, y3, y4, y5, y6, y7);

initial begin
    $dumpfile("gates_beh.vcd");
    $dumpvars(0, gates_beh_tb);

    a = 0; b = 0; #10;
    a = 0; b = 1; #10;
    a = 1; b = 0; #10;
    a = 1; b = 1; #10;

    $finish;
end

endmodule
```

8.5 RTL Schematic

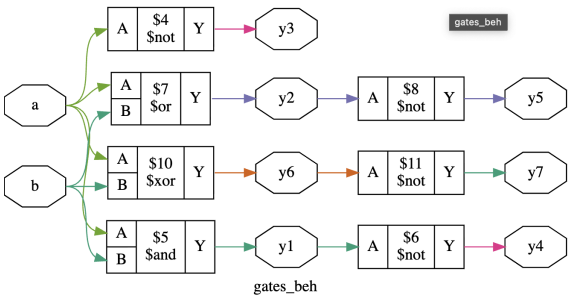


Figure 8.1: RTL schematic of basic gates (behavioral modeling)

8.6 Simulation Output

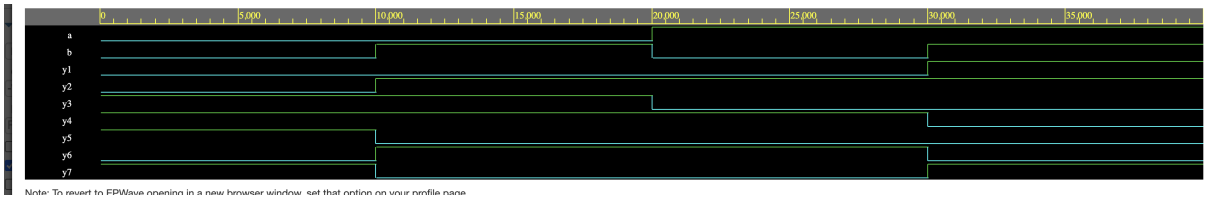


Figure 8.2: Simulation waveform of basic gates (behavioral modeling)

8.7 Observation Table

Table 8.1: Observed Outputs of Behavioral Gate Model

a	b	y1 (AND)	y2 (OR)	y3 (NOT a)	y4 (NAND)	y5 (NOR)	y6 (XOR)	y7 (XNOR)
0	0	0	0	1	1	1	0	1
0	1	0	1	1	1	0	1	0
1	0	0	1	0	1	0	1	0
1	1	1	1	0	0	0	0	1

8.8 Result

All seven basic gates were implemented using behavioral modeling and simulated successfully. The self-checking testbench applied all four input combinations and the outputs `y1` to `y7` matched the expected truth table.

Experiment 9
Mixed Dataflow and Procedural Modeling

9.1 Aim

To implement a circuit using both continuous assignment and procedural `always` blocks, with the outputs

$$Y_1 = A \,|\, B,\ Y_2 = Y_1 \oplus C,\ Y_3 = A \,\&\, B,\ Y_4 = Y_3 \,|\, C,\ Y_5 = A - B$$

where A, B and C are 4-bit inputs. Here $|$, & and $\oplus$ denote the bitwise OR, AND and XOR operations, and $A - B$ is arithmetic subtraction.

9.2 Apparatus

EDA Playground (Icarus Verilog, Yosys, EPWave).

9.3 Theory

Continuous assignments (`assign`) model combinational logic in the dataflow style, while `always @(*)` blocks model it procedurally. Both styles execute concurrently in the simulator. The first two outputs (Y_1, Y_2) are driven by `assign` statements, and the remaining outputs (Y_3, Y_4, Y_5) are driven by `always` blocks and are therefore declared as `reg`.

Net and Variable Types

Signals driven by `assign` must be of net type (`wire`), while signals assigned inside an `always` block must be of variable type (`reg`). The outputs Y_1 and Y_2 are therefore `wire`, and Y_3, Y_4, Y_5 are `reg`. As in Experiment 8, `reg` here does not create a flip-flop because the blocks are sensitive to all inputs (`@(*)`).

Dependencies Between Outputs

The output Y_2 depends on Y_1, and Y_4 depends on Y_3. In the continuous assignment, Y_2 is re-evaluated automatically whenever Y_1 changes. Inside the first `always` block, the blocking assignments execute in order, so Y_3 is updated before it is used to compute Y_4. Y_3 is also an output, so it appears in the waveform.

Bitwise Operations on 4-bit Buses

All operations act on 4-bit vectors, bit by bit. The subtraction $A - B$ is evaluated in 4 bits, so when $B > A$ the result wraps around modulo 16 (for example, $4 - 8 = 12$ in 4-bit arithmetic, which is 1100_2).

9.4 Verilog Code

Listing 9.1: Design: Mixed Dataflow and Procedural Modeling

```
`timescale 1ns/1ps

module gates(
  input wire [3:0] a,b,c,
  output wire [3:0] y1,y2,
  output reg [3:0] y3,y4,y5
  );

  // Dataflow (continuous assignments)
  assign y1 = a | b;
  assign y2 = y1 ^ c;

  // Procedural block 1
  always @(*) begin
    y3 = a & b;
    y4 = y3 | c;
  end

  // Procedural block 2
  always @(*) begin
    y5 = a - b;
  end
endmodule
```

Listing 9.2: Testbench: Mixed Dataflow and Procedural Modeling

```
`timescale 1ns/1ps

module gates_tb();
  reg  [3:0] a, b, c;
  wire [3:0] y1, y2, y3, y4, y5;

  integer errors;

  gates dut (a,b,c,y1,y2,y3,y4,y5);

  // Compares the outputs with expected values
  task check;
    reg [3:0] e1, e2, e3, e4, e5;
    begin
      e1 = a | b;
      e2 = e1 ^ c;
      e3 = a & b;
      e4 = e3 | c;
      e5 = a - b;
      if ({y1,y2,y3,y4,y5} !== {e1,e2,e3,e4,e5}) begin
        errors = errors + 1;
        $display("ERROR: a=%0d b=%0d c=%0d", a, b, c);
      end
    end
  endtask

  initial begin
    $dumpfile("gates.vcd");
```

```
29      $dumpvars(0, gates_tb);
30
31      $monitor("Time=%0t | a=%0d b=%0d c=%0d | y1=%0d y2=%0d y3=%0d y4=%0
            d y5=%0d",
32               $time, a, b, c, y1, y2, y3, y4, y5);
33
34      errors = 0;
35
36      a = 5;  b = 3;  c = 2;  #10; check;
37      a = 10; b = 10; c = 5;  #10; check;
38      a = 4;  b = 8;  c = 0;  #10; check;
39      a = 0;  b = 0;  c = 0;  #10; check;
40
41      if (errors == 0) $display("PASS: all 4 cases correct");
42      else             $display("FAIL: %0d errors", errors);
43
44      $finish;
45    end
46  endmodule
```

9.5 RTL Schematic

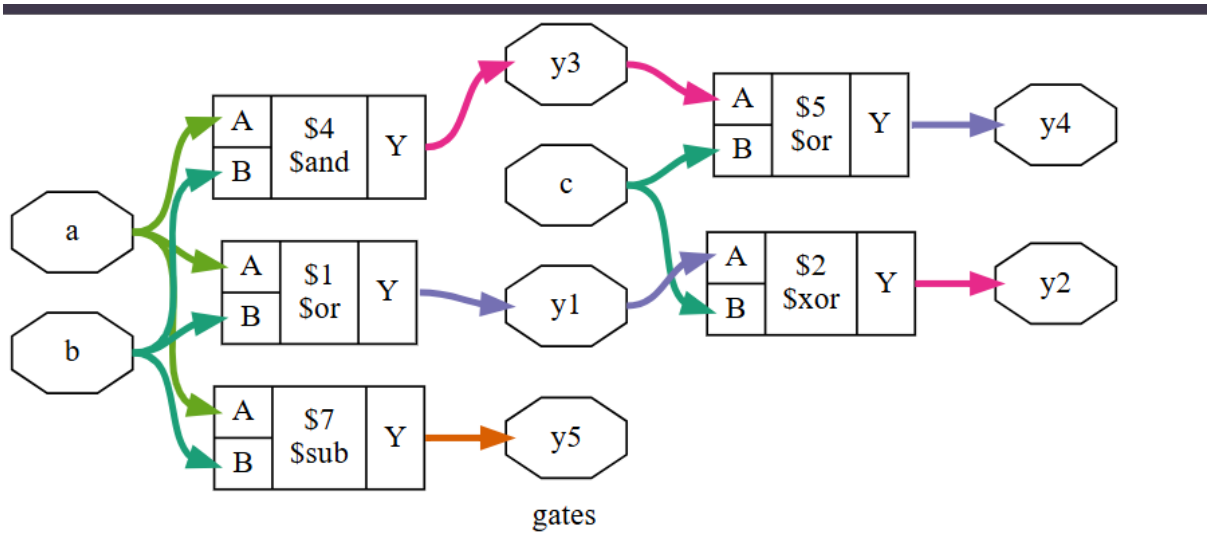


Figure 9.1: RTL schematic of the mixed dataflow and procedural design

9.6 Simulation Output

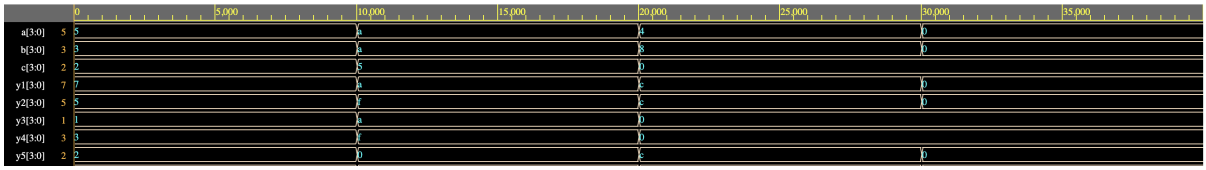


Figure 9.2: Simulation waveform of the mixed dataflow and procedural design

9.7 Observation Table

Table 9.1: Observed Outputs (all values in decimal, 4-bit)

a	b	c	y_1	y_2	y_3	y_4	y_5
5	3	2	7	5	1	3	2
10	10	5	10	15	10	15	0
4	8	0	12	12	0	0	12
0	0	0	0	0	0	0	0

For the case $a = 4$, $b = 8$, the subtraction gives $y_5 = 12$ because $4 - 8 = -4$ wraps around to $16 - 4 = 12$ in 4-bit arithmetic.

9.8 Result

A circuit mixing continuous assignments and procedural `always` blocks was simulated successfully. The self-checking testbench compared all five outputs with the expected values for four input sets and found no mismatch.

9.9 Conclusion

The experiment showed that dataflow and procedural styles can be combined in one module: `assign` statements drive `wire` outputs, and `always @(*)` blocks drive `reg` outputs. Both produce combinational logic, and outputs that depend on other outputs (Y_2 on Y_1, Y_4 on Y_3) are updated correctly. The 4-bit subtraction wraps around when $B > A$.

==========

Experiment 10
2-to-1 Multiplexer Using Behavioral Modeling

10.1 Aim

To implement a 2-to-1 multiplexer using behavioral modeling with an `if-else` statement inside an `always` block.

10.2 Apparatus

EDA Playground (Icarus Verilog, Yosys, EPWave).

10.3 Theory

A 2-to-1 multiplexer selects between two data inputs using a single select line. In this experiment the inputs are `a` and `b`, the select line is `select` and the output is `y`: $y = a$ when $\texttt{select} = 0$ and $y = b$ when $\texttt{select} = 1$, i.e.

$$y = \overline{\texttt{select}} \cdot a + \texttt{select} \cdot b$$

In behavioral modeling, the circuit is described by what it does rather than by its gates. The `always @(*)` block is re-evaluated whenever any input changes, which models combinational logic. The output is assigned inside the block, so it is declared as `reg`; this does not create a flip-flop. Because both branches of the `if-else` assign the output, no latch is inferred.

10.4 Verilog Code

Listing 10.1: Design: 2-to-1 Multiplexer (Behavioral, if-else)

```
// Code your design here
module mux(
  input wire a,b,select,
  output reg y

);

  always @(*) begin
    if(select==1'b0) begin
      y=a;
    end else begin
        y=b;

      end
    end
 endmodule
```

Listing 10.2: Testbench: 2-to-1 Multiplexer

```
`timescale 1ns/1ps

module mux_tb;

    reg a, b, select;
    wire y;

    // Instantiate 2:1 MUX
    mux uut (
        .a(a),
        .b(b),
      .select(select),
        .y(y)
    );

    initial begin
        $dumpfile("mux.vcd");
        $dumpvars(0, mux_tb);

        $monitor("Time=%0t | a=%b | b=%b | s=%b | y=%b",
                 $time, a, b, select, y);


        a = 0; b = 0; select = 0;
        #10;


        a = 1; b = 0; select = 0;
        #10;

        a = 0; b = 1; select = 1;
        #10;

        a = 1; b = 0; select = 1;
        #10;

        $finish;
    end

endmodule
```

10.5 RTL Schematic

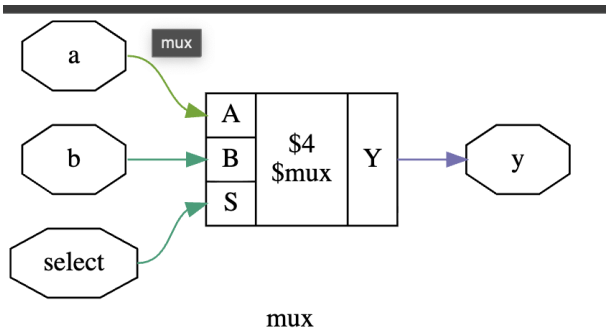


Figure 10.1: RTL schematic of the 2-to-1 multiplexer

10.6 Simulation Output

The testbench applies all eight input combinations at 10 ns intervals and compares `y` with the expected value `select ? b : a`. The console output is shown below (simulation time is printed in ps).

Listing 10.3: Console output of the simulation

```
Time=0 | a=0 | b=0 | s=0 | y=0
Time=10000 | a=1 | b=0 | s=0 | y=1
Time=20000 | a=0 | b=1 | s=0 | y=0
Time=30000 | a=1 | b=1 | s=0 | y=1
Time=40000 | a=0 | b=0 | s=1 | y=0
Time=50000 | a=1 | b=0 | s=1 | y=0
Time=60000 | a=0 | b=1 | s=1 | y=1
Time=70000 | a=1 | b=1 | s=1 | y=1
PASS: all 8 cases correct
```

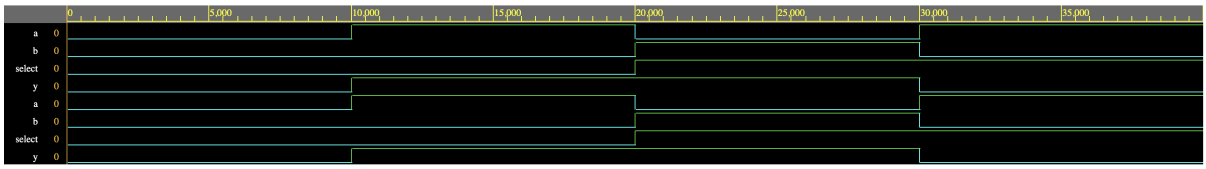


Figure 10.2: Simulation waveform of the 2-to-1 multiplexer

10.7 Observation Table

Table 10.1: Observed Output of the 2-to-1 Multiplexer

Time (ns)	a	b	select	y
0	0	0	0	0
10	0	0	1	1
20	0	1	0	0
30	0	1	1	1
40	1	0	0	0
50	1	0	1	0
60	1	1	0	1
70	1	1	1	1

10.8 Result

The 2-to-1 multiplexer was described using behavioral modeling and simulated successfully. The self-checking testbench applied all eight input combinations and the output `y` matched the expected value in every case.

10.9 Conclusion

A 2-to-1 multiplexer was implemented using an `if-else` statement inside an `always @(*)` block. The output followed `a` while `select` was 0 and `b` while `select` was 1, which matches the truth table. Since both branches assign the output, the design is purely combinational and no latch is inferre

Experiment 11
4-to-1 Multiplexer Using Behavioral Modeling

11.1 Aim

To implement a 4-to-1 multiplexer using behavioral modeling in three parts: (1) basic behavioral implementation using vector indexing, (2) using if-else statements, and (3) using a case statement.

11.2 Apparatus

EDA Playground (Icarus Verilog, Yosys).

11.3 Theory

A 4-to-1 multiplexer selects one of four data inputs I_0 to I_3 using a 2-bit select line S_1S_0 and passes it to the output Y:

$$Y = \overline{S_1}\,\overline{S_0}\,I_0 + \overline{S_1}\,S_0\,I_1 + S_1\,\overline{S_0}\,I_2 + S_1\,S_0\,I_3$$

In behavioral modeling, the circuit is described by what it does rather than by its gates. The `always @(*)` block is re-evaluated whenever any input changes, which models combinational logic. Because every branch assigns the output, no latch is inferred.

Table 11.1: Function table of the 4-to-1 multiplexer

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

11.4 Verilog Code

Listing 11.1: Design: 4-to-1 Multiplexer (Part 1, Part 2, Part 3)

```
// Part 1: Basic behavioral implementation using vector indexing
module mux4x1_m1(input [3:0] I, input [1:0] S, output reg Y1);
    always @(*) begin
        Y1 = I[S];
    end
endmodule

// Part 2: Using if-else statements
module mux4x1_m2(input [3:0] I, input [1:0] S, output reg Y2);
```

```
    always @(*) begin
        if (S == 2'b00)
            Y2 = I[0];
        else if (S == 2'b01)
            Y2 = I[1];
        else if (S == 2'b10)
            Y2 = I[2];
        else
            Y2 = I[3];
    end
endmodule

// Part 3: Using case statement
module mux4x1_m3(input [3:0] I, input [1:0] S, output reg Y3);
    always @(*) begin
        case (S)
            2'b00: Y3 = I[0];
            2'b01: Y3 = I[1];
            2'b10: Y3 = I[2];
            2'b11: Y3 = I[3];
            default: Y3 = 1'b0;
        endcase
    end
endmodule
```

Listing 11.2: Testbench: 4-to-1 Multiplexer

```
`timescale 1ns/1ps

module mux4x1_beh_tb;

reg [3:0] I;
reg [1:0] S;
wire Y1, Y2, Y3;
reg expected;
integer i, errors;

mux4x1_m1 uut1(I, S, Y1);
mux4x1_m2 uut2(I, S, Y2);
mux4x1_m3 uut3(I, S, Y3);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, mux4x1_beh_tb);
    errors = 0;

    for (i = 0; i < 64; i = i + 1) begin
        {S, I} = i;
        expected = I[S];
        #10;
        if (Y1 !== expected || Y2 !== expected || Y3 !== expected)
           begin
            errors = errors + 1;
            $display("FAIL: S=%b I=%b exp=%b Y1=%b Y2=%b Y3=%b",
                     S, I, expected, Y1, Y2, Y3);
        end else
            $display("PASS: S=%b I=%b Y=%b", S, I, Y1);
    end
```

```
31
32      if (errors == 0) $display("ALL TESTS PASSED");
33      else             $display("%0d TESTS FAILED", errors);
34      $finish;
35  end
36
37  endmodule
```

11.5 Results

11.5.1 Simulation Output

The testbench verified all 64 combinations of the 4-bit input and 2-bit select line. All three modules produced the expected output with no failures.

Listing 11.3: Trimmed simulation output

```
1  PASS: S=00 I=0000 Y=0
2  PASS: S=00 I=0001 Y=1
3  ...
4  PASS: S=11 I=1111 Y=1
5  ALL TESTS PASSED
```

11.5.2 Synthesis Result

The three modules were synthesized using Yosys. Although the coding styles differ, all implement the same 4-to-1 MUX function.

Table 11.2: Yosys synthesis results

Module	Cells
`mux4x1_m1` (indexing)	1 `$shiftx`
`mux4x1_m2` (if-else)	3 `$mux`, 2 `$eq`, 1 `$logic_not`
`mux4x1_m3` (case)	1 `$pmux`, 3 `$eq`, 1 `$logic_not`

11.6 Observations

- All three styles produced identical outputs for all 64 test cases.
- Vector indexing gives the most compact synthesized structure.
- The if-else style generates a priority MUX structure.
- The case style generates a parallel MUX structure.
- No latches were inferred in any implementation.

Experiment 12
3-to-8 Decoder with Enable

12.1 Aim

To design and implement a 3-to-8 decoder with an enable input. The decoder operates when the enable input is high.

12.2 Apparatus

EDA Playground (Icarus Verilog, Yosys, EPWave).

12.3 Theory

A 3-to-8 decoder activates exactly one of its eight output lines depending on the 3-bit input. When the enable input is low, all outputs remain low.

Table 12.1: Function Table of 3-to-8 Decoder with Enable

Enable	In[2:0]	Out[7:0]
0	XXX	00000000
1	000	00000001
1	001	00000010
1	010	00000100
1	011	00001000
1	100	00010000
1	101	00100000
1	110	01000000
1	111	10000000

12.4 Verilog Code

Listing 12.1: Design: 3-to-8 Decoder with Enable

```
module decoder38(
    input enable,
    input [2:0] in,
    output reg [7:0] out
);

always @(*) begin
    if (enable) begin
        case (in)
            3'b000: out = 8'b00000001;
            3'b001: out = 8'b00000010;
```

```
            3'b010: out = 8'b00000100;
            3'b011: out = 8'b00001000;
            3'b100: out = 8'b00010000;
            3'b101: out = 8'b00100000;
            3'b110: out = 8'b01000000;
            3'b111: out = 8'b10000000;
            default: out = 8'b00000000;
        endcase
    end else begin
        out = 8'b00000000;
    end
end

endmodule
```

Listing 12.2: Testbench: 3-to-8 Decoder with Enable

```
`timescale 1ns/1ps

module decoder38_tb;

reg enable;
reg [2:0] in;
wire [7:0] out;

decoder38 uut(enable, in, out);

integer i;

initial begin
    $dumpfile("decoder38.vcd");
    $dumpvars(1, decoder38_tb);

    $monitor("Time=%0t | enable=%b | in=%b | out=%b", $time, enable, in
        , out);

    enable = 0;
    for (i = 0; i < 8; i = i + 1) begin
        in = i;
        #10;
    end

    enable = 1;
    for (i = 0; i < 8; i = i + 1) begin
        in = i;
        #10;
    end

    $finish;
end

endmodule
```

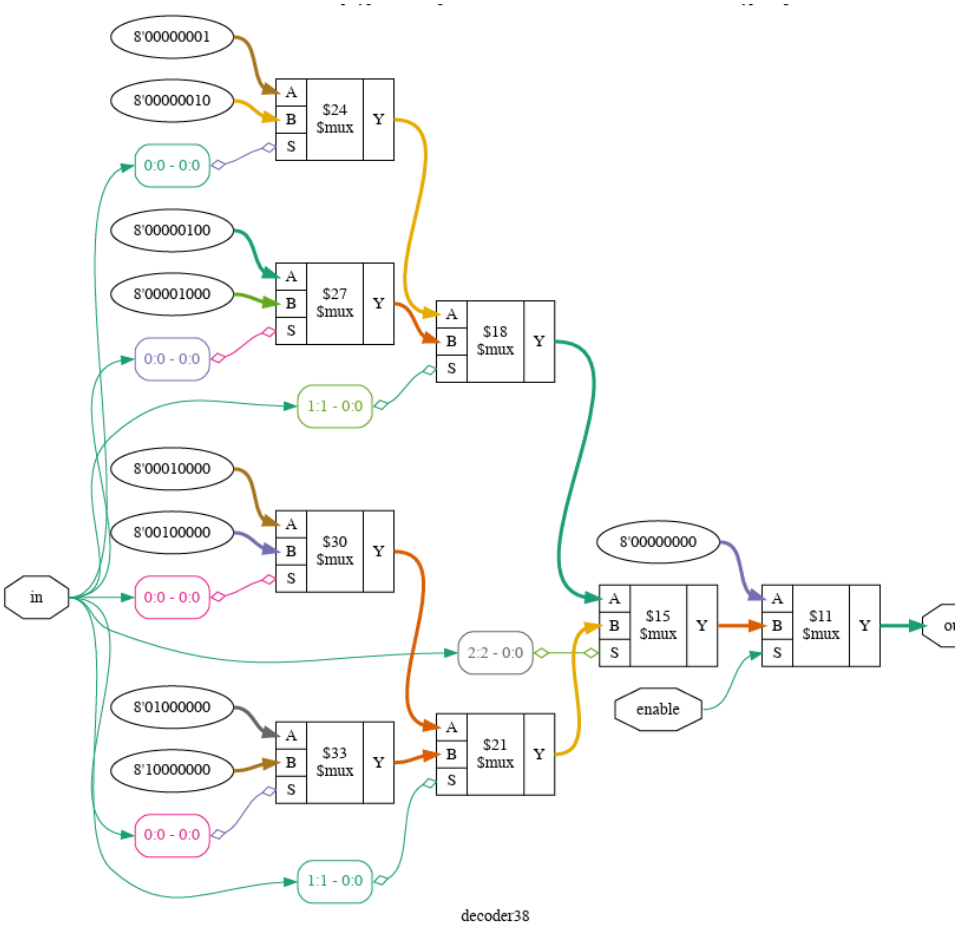


Figure 12.1: Yosys output

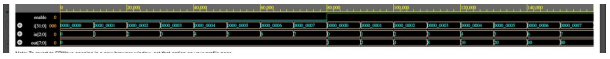


Figure 12.2: waveform

Experiment 13
Multiplication and Division Using Shift Operations

13.1 Aim

To implement multiplication using the left-shift operation and division using the right-shift operation.

13.2 Apparatus

EDA Playground (Icarus Verilog, Yosys, EPWave).

13.3 Theory

Shifting a binary number left by n positions multiplies it by 2^n, and shifting it right by n positions divides it by 2^n (integer division, discarding the remainder). Shift operators therefore give a hardware-efficient way of multiplying or dividing by powers of two.

$$A \ll n = A \times 2^n, \qquad A \gg n = \left\lfloor \frac{A}{2^n} \right\rfloor$$

13.4 Verilog Code

Listing 13.1: Design: Multiplication and Division Using Shift

```
module shift_mul_div(
    input [7:0] a,
    input [2:0] n,
    output [15:0] mul,
    output [7:0]  div
);

assign mul = {8'b0, a} << n;    // a * 2^n
assign div = a >> n;            // a / 2^n

endmodule
```

Listing 13.2: Testbench: Multiplication and Division Using Shift

```
`timescale 1ns/1ps

module shift_mul_div_tb;

reg [7:0] a;
reg [2:0] n;
```

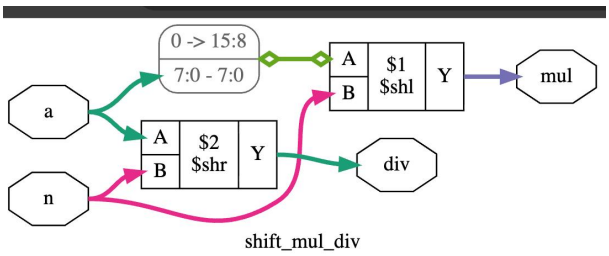


Figure 13.1: Yosys output

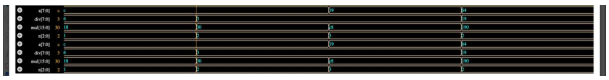


Figure 13.2: Waveform

```
7  wire [15:0] mul;
8  wire [7:0] div;
9
10 shift_mul_div uut(a, n, mul, div);
11
12 initial begin
13     $dumpfile("shift_mul_div.vcd");
14     $dumpvars(0, shift_mul_div_tb);
15
16     $monitor("Time=%0t | a=%d n=%d | mul=%d div=%d", $time, a, n, mul,
           div);
17
18     a = 8'd12;  n = 3'd1; #10;
19     a = 8'd12;  n = 3'd2; #10;
20     a = 8'd25;  n = 3'd3; #10;
21     a = 8'd100; n = 3'd2; #10;
22
23     $finish;
24 end
25
26 endmodule
```

Experiment 14
Implementation of a Decoder Using Shift Operations

14.1 Aim

To design and implement a 3-to-8 decoder with enable using the left-shift operation.

14.2 Apparatus

EDA Playground (Icarus Verilog, Yosys, EPWave).

14.3 Theory

A decoder output has exactly one bit set, at the position given by the binary input. This is equivalent to shifting the constant `8'b00000001` left by the value of the input. When the enable input is low, the output is forced to zero.

14.4 Verilog Code

Listing 14.1: Design: Decoder Using Shift Operator

```
// Code your design here
module decoder(
    input wire enable,
  input wire [2:0]in,
  output reg [7:0]out

);

  always @(*)begin
    out =(enable)?(8'b0000_0001 << in):8'b0000_0000;


  end
endmodule
```

Listing 14.2: Testbench: Decoder Using Shift Operator

```
`timescale 1ns/1ps

module decoder_tb();
  reg enable;
  reg [2:0] in;
  wire [7:0] out;

```

```
8   decoder uut(enable,in,out);
9
10  integer i;
11
12  initial begin
13    $dumpfile("decoder.vcd");
14    $dumpvars(1, decoder_tb);
15
16    $monitor("Time=%0t | enable=%b | in=%b (%0d) | out=%b",
17             $time, enable, in, in, out);
18    enable = 0; // Test disabled state (enable = 0)
19    for (i = 0; i < 8; i = i + 1) begin
20      in = i;
21      #10;
22    end
23
24    enable = 1;
25    for (i = 0; i < 8; i = i + 1) begin
26      in = i;
27      #10;
28    end
29
30    $finish;
31  end
32 endmodule
```

14.5 RTL Schematic

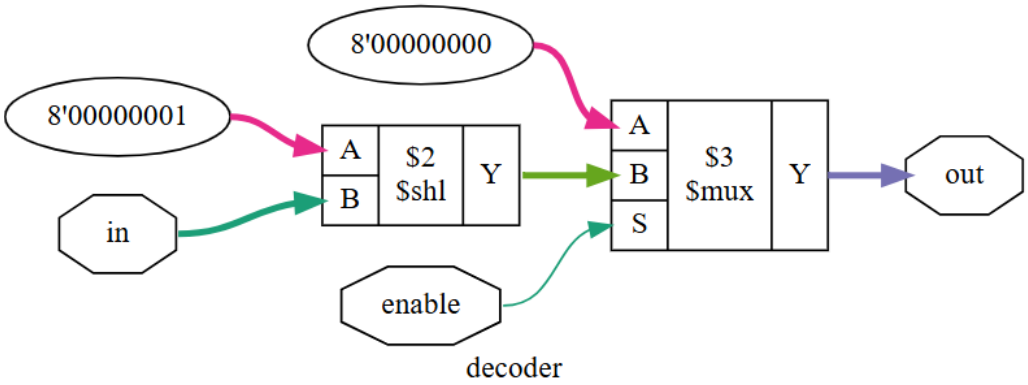


Figure 14.1: RTL schematic of the shift-based decoder

14.6 Simulation Output

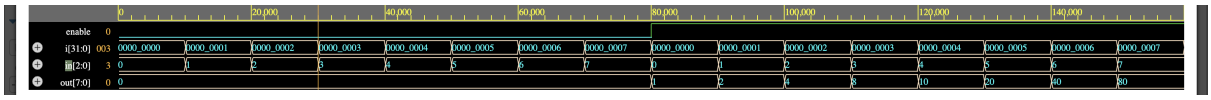


Figure 14.2: Simulation waveform of the shift-based decoder

14.7 Conclusion

With enable low the output stays at `00000000`; with enable high, exactly one output bit is set at the position given by the input.

Class Task: Design of a 4-bit Arithmetic Logic Unit (ALU)

14.8 Aim

To design a 4-bit ALU that performs different operations based on a 3-bit opcode.

14.9 Apparatus

EDA Playground (Icarus Verilog, Yosys, EPWave).

14.10 Theory

An ALU is a combinational block that performs arithmetic and logical operations on its operands. The operation is selected by the opcode and the unit operates only when the enable input is high; otherwise the output is zero.

Table 14.1: ALU Operations

Opcode	Operation
000	A + B
001	A − B
010	A AND B
011	A OR B
100	A XOR B
101	NOT A
110	Shift left A by 1 bit
111	Shift right A by 1 bit

14.11 Verilog Code

Listing 14.3: Design: 4-bit ALU

```
// Code your design here
module alu(
  input wire [3:0] a, b,
  input wire enable,
  input wire [2:0] opcode,
  output reg [3:0] out
);

  always @(*) begin
    if (!enable) begin
      out = 4'b0000;
    end else begin
      case (opcode)
```

```
        3'b000: out = a + b;
        3'b001: out = a - b;
        3'b010: out = a & b;
        3'b011: out = a | b;
        3'b100: out = a ^ b;
        3'b101: out = ~a;
        3'b110: out = a << 1;
        3'b111: out = a >> 1;
        default: out = 4'b0000;
      endcase
    end
  end
endmodule
```

Listing 14.4: Testbench: 4-bit ALU

```
`timescale 1ns/1ps;

module alu_tb();
  reg [3:0] a;
  reg [3:0] b;
  reg enable;
  reg [2:0] opcode;
  wire [3:0] out;

  alu uut (a,b,enable,opcode,out
  );

  integer i;

  initial begin
    $dumpfile("alu.vcd");
    $dumpvars(1, alu_tb);

    $monitor("Time=%0t | A=%b  | B=%b | Opcode=%b | Out=%b ",
             $time, a, b, opcode, out);

    a = 4'b1010;
    b = 4'b0011;

    for (i = 0; i < 8; i = i + 1) begin
      opcode = i;
      #10
    end

    $finish;
  end
endmodule
```

14.12 RTL Schematic

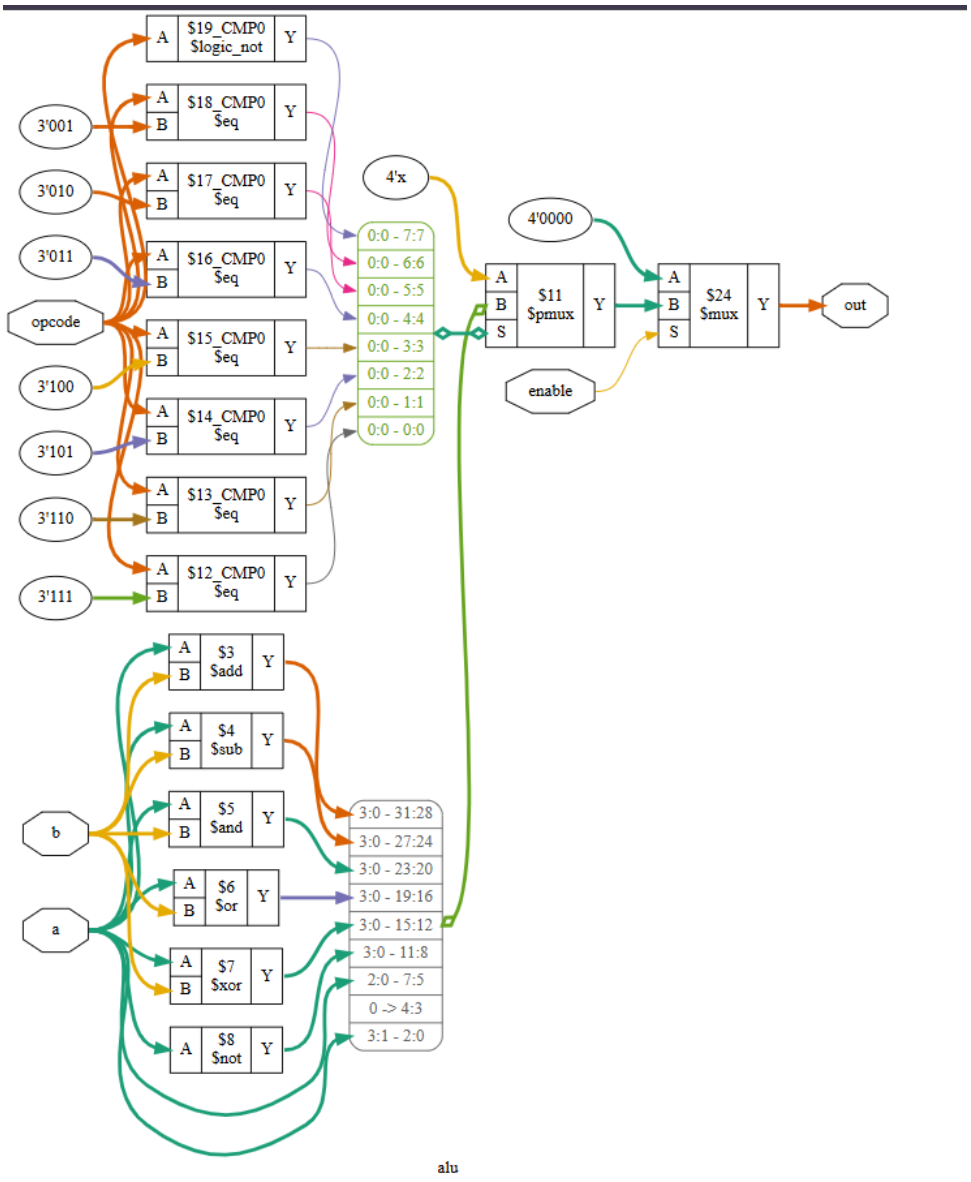


Figure 14.3: RTL schematic of the 4-bit ALU

14.13 Simulation Output

```
VCD info: dumpfile alu.vcd opened for output.
Time=0 | A=1010  | B=0011 | Opcode=000 | Out=1101
Time=10000 | A=1010  | B=0011 | Opcode=001 | Out=0111
Time=20000 | A=1010  | B=0011 | Opcode=010 | Out=0010
Time=30000 | A=1010  | B=0011 | Opcode=011 | Out=1011
Time=40000 | A=1010  | B=0011 | Opcode=100 | Out=1001
Time=50000 | A=1010  | B=0011 | Opcode=101 | Out=0101
Time=60000 | A=1010  | B=0011 | Opcode=110 | Out=0100
Time=70000 | A=1010  | B=0011 | Opcode=111 | Out=0101
testbench.sv:30: $finish called at 80000 (1ps)
Done
```

Figure 14.4: Console output of the ALU testbench

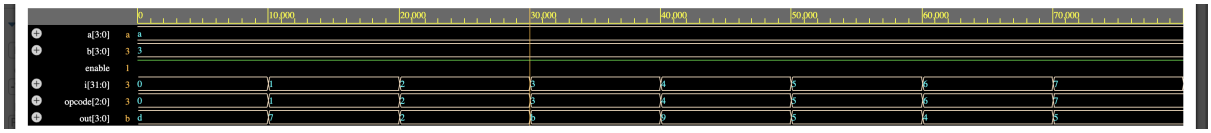


Figure 14.5: Simulation waveform of the 4-bit ALU

Table 14.2: ALU Results for A = 1010, B = 0011

Opcode	Operation	Output
000	A + B	1101
001	A − B	0111
010	A AND B	0010
011	A OR B	1011
100	A XOR B	1001
101	NOT A	0101
110	$A \ll 1$	0100
111	$A \gg 1$	0101

14.14 Conclusion

The 4-bit ALU correctly performs all eight operations for the chosen operands, and the simulated outputs agree with the expected results.

Experiment 15
Basic Logic Gates Using OpenLane

15.1 Aim

To implement the basic logic gates AND, OR, NOT, NAND, NOR, XOR and XNOR in Verilog HDL and to carry them through the OpenLane user design flow: simulation, waveform viewing and RTL schematic generation.

15.2 Apparatus

OpenLane environment (Ubuntu on WSL), Icarus Verilog (`iverilog`, `vvp`), GTKWave, Yosys and Graphviz (`dot`).

15.3 Theory

In the OpenLane user environment every design is kept in its own folder under `~/OpenLaneUser/designs`. The Verilog module and its testbench are compiled with Icarus Verilog, the generated VCD file is viewed in GTKWave, and the RTL schematic is generated by Yosys (`prep` command) and converted to a PDF using Graphviz.

The design is purely combinational and has two inputs, `a` and `b`, and eight outputs: `y_or`, `y_nor`, `y_and`, `y_nand`, `y_xor`, `y_xnor`, `y_nota` and `y_notb`. The gates are described by continuous assignments using the bitwise operators `&`, `|`, `~` and `^`. NAND, NOR and XNOR are obtained by inverting the output of AND, OR and XOR. There is no clock or reset, so the outputs change as soon as the inputs change.

Table 15.1: Truth Table of Basic Gates

A	B	AND	OR	NAND	NOR	XOR	XNOR	NOT A	NOT B
0	0	0	0	1	1	0	1	1	1
0	1	0	1	1	0	1	0	1	0
1	0	0	1	1	0	1	0	0	1
1	1	1	1	0	0	0	1	0	0

15.4 Verilog Code

Listing 15.1: Design: Basic Logic Gates

```
1 `timescale 1ns/1ps
2 module gates(
3   input a,b,
4   output y_or,y_nor,y_and,y_nand,y_xor,y_xnor,y_nota,y_notb
5 );
6   assign y_or=a|b;
```

```
  assign y_nor=~(a|b);
  assign y_and=a & b;
  assign y_nand=~(a & b);
  assign y_xor=a^b;
  assign y_xnor=~(a^b);
  assign y_nota=~a;
  assign y_notb=~b;
endmodule
```

Listing 15.2: Testbench: Basic Logic Gates

```
`timescale 1ns/1ps

module gates_tb();
    reg a, b;
    wire y_or, y_nor, y_and, y_nand;
    wire y_xor, y_xnor, y_nota, y_notb;

    gates uut(
        a, b,
        y_or, y_nor, y_and, y_nand,
        y_xor, y_xnor, y_nota, y_notb
    );

    initial begin
        $dumpfile("gates.vcd");
        $dumpvars(1, gates_tb);

        $monitor("Time=%0t | a=%b b=%b | OR=%b NOR=%b AND=%b NAND=%b
           XOR=%b XNOR=%b NOT_A=%b NOT_B=%b",
                 $time, a, b,
                 y_or, y_nor, y_and, y_nand,
                 y_xor, y_xnor, y_nota, y_notb);

        // Test cases
        a = 0; b = 0;
        #10;
        a = 0; b = 1;
        #10;
        a = 1; b = 0;
        #10;
        a = 1; b = 1;
        #10;
        $finish;
    end
endmodule
```

15.5 Simulation Output

```
sachin@Sachin-PC:~/OpenLaneUser/designs/T_folder$ explorer.exe gates_rtl.pdf
sachin@Sachin-PC:~/OpenLaneUser/designs/T_folder$ nano tb_bg.v
sachin@Sachin-PC:~/OpenLaneUser/designs/T_folder$ iverilog -o bg_sim bg.v tb
_bg.v
vvp bg_sim
VCD info: dumpfile gates.vcd opened for output.
Time=0 | a=0 b=0 | OR=0 NOR=1 AND=0 NAND=1 XOR=0 XNOR=1 NOT_A=1 NOT_B=1
Time=10000 | a=0 b=1 | OR=1 NOR=0 AND=0 NAND=1 XOR=1 XNOR=0 NOT_A=1 NOT_B=0
Time=20000 | a=1 b=0 | OR=1 NOR=0 AND=0 NAND=1 XOR=1 XNOR=0 NOT_A=0 NOT_B=1
Time=30000 | a=1 b=1 | OR=1 NOR=0 AND=1 NAND=0 XOR=0 XNOR=1 NOT_A=0 NOT_B=0
tb_bg.v:26: $finish called at 40000 (1ps)
sachin@Sachin-PC:~/OpenLaneUser/designs/T_folder$
```

Figure 15.1: Console output of the basic logic gates simulation (Icarus Verilog)

15.6 RTL Schematic

```
sachin@Sachin-PC:~/OpenLaneUser/designs/T_folder$ dot -Tpdf gates_rtl.dot -o
 gates_rtl.pdf
sachin@Sachin-PC:~/OpenLaneUser/designs/T_folder$ explorer.exe gates_rtl.pdf
```

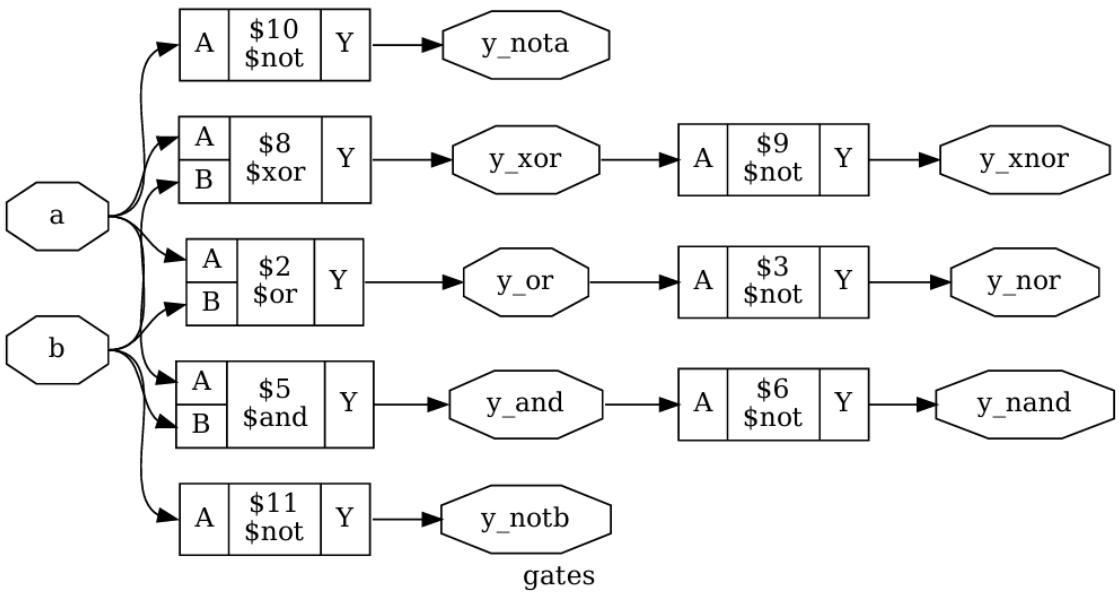


Figure 15.2: RTL schematic of basic logic gates generated by Yosys

15.7 Waveform

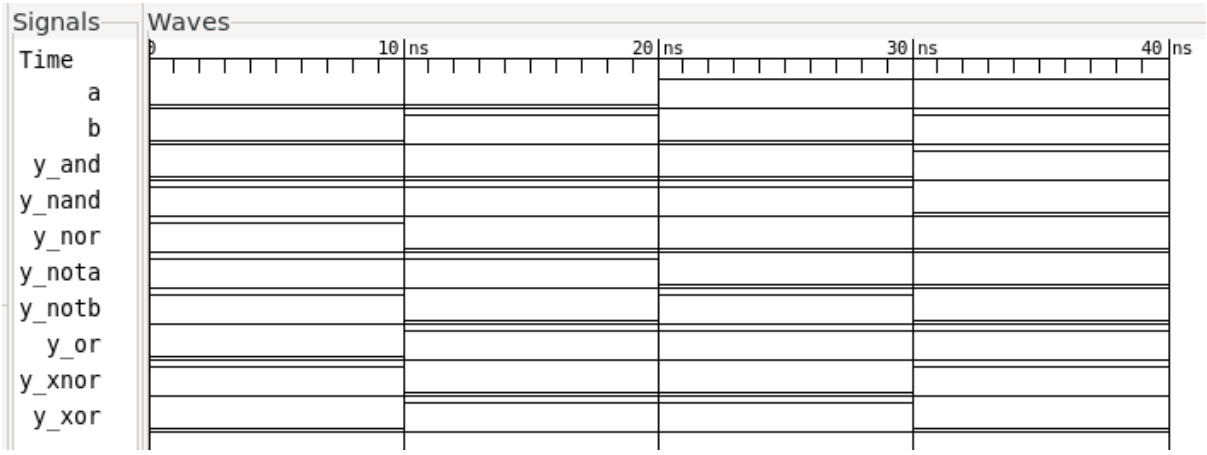


Figure 15.3: GTKWave waveform of basic logic gates

15.8 Result

The basic logic gates were simulated successfully. For all four input combinations (00, 01, 10 and 11) the outputs of AND, OR, NOT, NAND, NOR, XOR and XNOR matched the truth table, and the waveform and RTL schematic were generated through the OpenLane user flow.

15.9 Conclusion

The gates were described with continuous assignments and verified by applying every input combination for 10 ns each. Since the design is combinational, each output changed at the same instant as the input, with no delay in the simulation. NAND, NOR and XNOR behaved as the complements of AND, OR and XOR respectively. Yosys implemented the design as a set of basic logic cells (AND, OR, XOR and NOT), which matches the Verilog description.

Experiment 16
NAND Gate and Gated D Latch Using OpenLane

16.1 Aim

To implement a NAND gate in Verilog HDL, to use NAND gates to build a clocked (gated) D latch, and to simulate both through the OpenLane user design flow.

16.2 Apparatus

OpenLane environment (Ubuntu on WSL), Icarus Verilog (`iverilog`, `vvp`), GTKWave, Yosys and Graphviz (`dot`).

16.3 Theory

The NAND gate is a universal gate: its output is 0 only when both inputs are 1. Any other logic function, including a storage element, can be built from NAND gates alone.

A gated D latch has a data input `d`, an enable input `en`, and complementary outputs `q` and `qbar`. It is built from five NAND gates, connected as shown in Fig. 16.1:

- one NAND gate with both inputs tied to `d` acts as an inverter and produces $\overline{d}$,
- two NAND gates gate the data with the enable: $n_1 = \overline{d \cdot en}$ and $n_2 = \overline{\overline{d} \cdot en}$,
- two cross-coupled NAND gates form the SR latch: $q = \overline{n_1 \cdot qbar}$ and $qbar = \overline{n_2 \cdot q}$.

When `en` $= 1$ the latch is transparent and `q` follows `d`. When `en` $= 0$ both n_1 and n_2 are 1, so the cross-coupled pair holds its previous value. Because the latch is level sensitive, `q` can change at any time while `en` is high. The latch starts in an unknown state, so the testbench enables it first to set a known value.

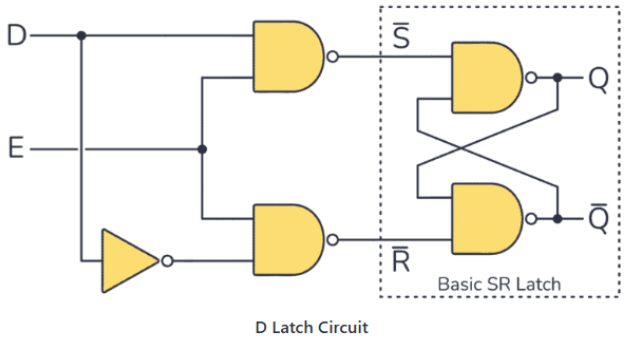


Figure 16.1: Gate-level circuit of the NAND-based gated D latch

16.4 Verilog Code

Listing 16.1: Design: NAND Gate and Gated D Latch

```
`timescale 1ns/1ps

// Basic NAND gate
module nand_gate(
  input a, b,
  output y
);
  assign y = ~(a & b);
endmodule

// Gated D latch built from NAND gates only
module d_latch(
  input d, en,
  output q, qbar
);
  wire dn;
  wire n1, n2;

  nand_gate g0(d,d,dn);   // inverter
  nand_gate g1(d,en,n1);
  nand_gate g2(dn,en,n2);
  nand_gate g3(n1,qbar,q);    // cross-coupled pair
  nand_gate g4(n2, q, qbar);
endmodule
```

Listing 16.2: Testbench: NAND Gate and Gated D Latch

```
`timescale 1ns/1ps

module dlatch_tb();
    reg d, en;
    wire y_nand, q, qbar;

    nand_gate u_nand(d,en,y_nand);
     d_latch uut(d, en, q,qbar);

    initial begin
        // VCD generation
        $dumpfile("dlatch.vcd");
        $dumpvars(1, dlatch_tb);

        // Display output whenever any signal changes
        $monitor("Time=%0t | d=%b en=%b | NAND=%b | Q=%b Qbar=%b",
                 $time, d, en, y_nand, q, qbar);

        // Enable first to put the latch in a known state
        d = 0; en = 1;  #10;   // Q = 0 (transparent)
        d = 1; en = 1;  #10;   // Q = 1 (transparent)
        d = 1; en = 0;  #10;   // hold 1
        d = 0; en = 0;  #10;   // hold 1, d ignored
        d = 0; en = 1;  #10;   // Q = 0 (transparent)
        d = 1; en = 0;  #10;   // hold 0, d ignored
        d = 1; en = 1;  #10;   // Q = 1 (transparent)
        d = 0; en = 0;  #10;   // hold 1
```

```
28          $finish;
29      end
30  endmodule
```

16.5 Simulation Output

```
VCD info: dumpfile dlatch.vcd opened for output.
Time=0 | d=0 en=1 | NAND=1 | Q=0 Qbar=1
Time=10000 | d=1 en=1 | NAND=0 | Q=1 Qbar=0
Time=20000 | d=1 en=0 | NAND=1 | Q=1 Qbar=0
Time=30000 | d=0 en=0 | NAND=1 | Q=1 Qbar=0
Time=40000 | d=0 en=1 | NAND=1 | Q=0 Qbar=1
Time=50000 | d=1 en=0 | NAND=1 | Q=0 Qbar=1
Time=60000 | d=1 en=1 | NAND=0 | Q=1 Qbar=0
Time=70000 | d=0 en=0 | NAND=1 | Q=1 Qbar=0
tb_dl.v:28: $finish called at 80000 (1ps)
```

Figure 16.2: Console output of the NAND gate and D latch simulation (Icarus Verilog)

16.6 RTL Schematic

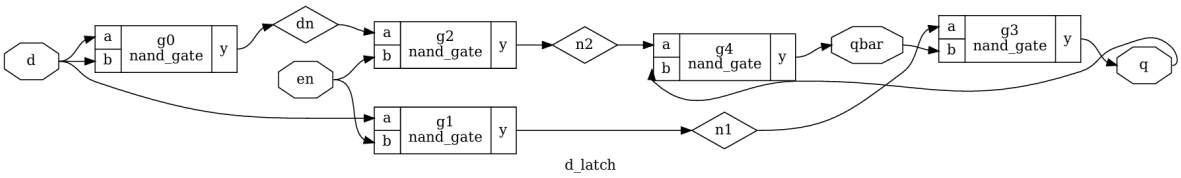


Figure 16.3: RTL schematic of the NAND-based D latch generated by Yosys

16.7 Waveform

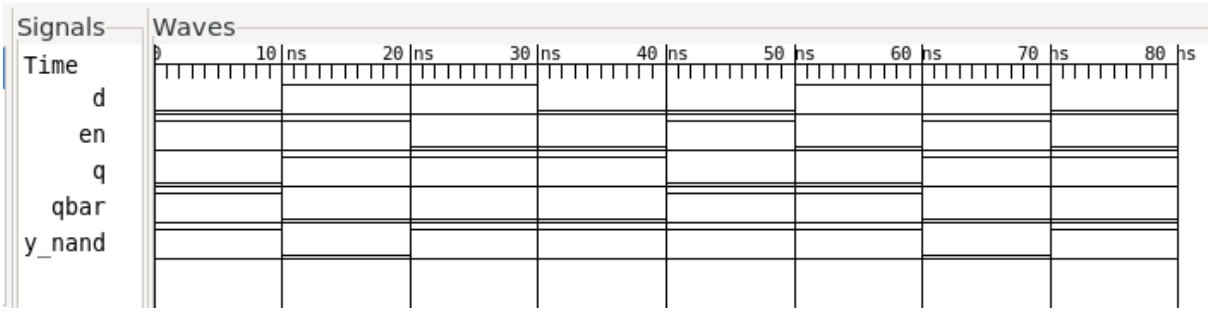


Figure 16.4: GTKWave waveform of the NAND gate and gated D latch

16.8 Observation Table

Table 16.1: Observed Behavior of the NAND Gate and Gated D Latch

Time (ns)	`d`	`en`	NAND	Q	Qbar	Remark
0	0	1	1	0	1	Transparent, Q follows d
10	1	1	0	1	0	Transparent, Q follows d
20	1	0	1	1	0	Latched, Q holds 1
30	0	0	1	1	0	Latched, d change ignored
40	0	1	1	0	1	Transparent, Q follows d
50	1	0	1	0	1	Latched, d change ignored
60	1	1	0	1	0	Transparent, Q follows d
70	0	0	1	1	0	Latched, Q holds 1

16.9 Result

The NAND gate and the NAND-based gated D latch were simulated successfully. The NAND output matched its truth table for all four input combinations. The latch followed `d` while `en` was high and held its value while `en` was low.

16.10 Conclusion

The gated D latch was built from five NAND gates, which shows that NAND is a universal gate and that a memory element can be made from it. With `en` = 1 the output `q` follows `d`. With `en` = 0 the cross-coupled NAND pair holds the stored value and ignores changes in `d`. The outputs `q` and `qbar` were always complementary after the first enable. Yosys implemented the design as NAND-type logic cells with a feedback loop, which matches the Verilog description.

Experiment 17
Clocked D Latch Using Blocking and Non-Blocking Assignments

17.1 Aim

To implement a clocked D latch using blocking (`=`) and non-blocking (`<=`) assignments and to compare their behavior.

17.2 Apparatus

OpenLane environment (Ubuntu on WSL), Icarus Verilog (`iverilog`, `vvp`), GTKWave, Yosys and Graphviz (`dot`).

17.3 Theory

A clocked D latch is a level-sensitive storage element with a data input `d`, a clock (enable) input `clk` and an output `q`. While `clk` = 1 the latch is transparent and `q` follows `d`. While `clk` = 0 the latch holds its last value and ignores `d`. Before the first transparent period the value of `q` is unknown (`x`).

Table 17.1: Function Table of the Clocked D Latch

`clk`	`d`	q_{next}	Operation
0	X	q	Hold (latched)
1	0	0	Transparent, reset
1	1	1	Transparent, set

Verilog has two procedural assignment operators, and the two designs differ only in which one they use.

Table 17.2: Blocking and Non-Blocking Assignments

Property	Blocking (`=`)	Non-blocking (`<=`)
Execution	Statements run one after another	Right-hand sides are all evaluated first
Update of left side	Immediately	At the end of the time step
Next statement waits	Yes	No
Usual use	Combinational logic	Sequential logic and latches

The latch has one assignment in one `always` block, and its sensitivity list is complete. Both versions should therefore give the same output at the end of every time step. Non-blocking assignment is still the recommended style for latches, because it keeps the same coding rule for every storage element and avoids race conditions when latches are cascaded or read by other blocks.

17.4 Verilog Code

Listing 17.1: Design: Clocked D Latch with Blocking and Non-Blocking Assignments

```
`timescale 1ns/1ps

// D latch using blocking assignment
module dlatch_blk(
  input clk, d,
  output reg q
);
  always @(*) begin
    if (clk)
      q = d;         // blocking
  end
endmodule

// D latch using non-blocking assignment
module dlatch_nb(
  input clk, d,
  output reg q
);
  always @(*) begin
    if (clk)
      q <= d;        // non-blocking
  end
endmodule
```

Listing 17.2: Testbench: Blocking and Non-Blocking D Latch

```
`timescale 1ns/1ps

module dlatch_tb();
    reg clk, d;
    wire q_blk, q_nb;

    dlatch_blk u_blk(.clk(clk), .d(d), .q(q_blk));
    dlatch_nb  u_nb (.clk(clk), .d(d), .q(q_nb));

    // Clock: period 20 ns
    always #10 clk = ~clk;

    initial begin
        // VCD generation
        $dumpfile("dlatch_bnb.vcd");
        $dumpvars(1, dlatch_tb);

        // Display output whenever any signal changes
        $monitor("Time=%0d | clk=%b d=%b | Q_blk=%b Q_nb=%b | Match=%b"
            ,
                 $time, clk, d, q_blk, q_nb, (q_blk === q_nb));

        clk = 0; d = 0;
        #5  d = 1;   // clk low : ignored
        #10 d = 0;   // clk high: Q follows d
        #10 d = 1;   // clk low : ignored
        #10 d = 0;   // clk high: Q follows d
        #10 d = 1;   // clk low : ignored
```

```
28          #10 d = 0;    // clk high: Q follows d
29          #10 d = 1;    // clk low : ignored
30          #10 d = 0;    // clk high: Q follows d
31          #10 d = 1;    // clk low : ignored
32          #10 d = 0;    // clk high: Q follows d
33          #15;
34          $finish;
35      end
36  endmodule
```

17.5 Simulation Output

```
VCD info: dumpfile dlatch_bnb.vcd opened for output.
Time=0 | clk=0 d=0 | Q_blk=x Q_nb=x | Match=1
Time=5 | clk=0 d=1 | Q_blk=x Q_nb=x | Match=1
Time=10 | clk=1 d=1 | Q_blk=1 Q_nb=1 | Match=1
Time=15 | clk=1 d=0 | Q_blk=0 Q_nb=0 | Match=1
Time=20 | clk=0 d=0 | Q_blk=0 Q_nb=0 | Match=1
Time=25 | clk=0 d=1 | Q_blk=0 Q_nb=0 | Match=1
Time=30 | clk=1 d=1 | Q_blk=1 Q_nb=1 | Match=1
Time=35 | clk=1 d=0 | Q_blk=0 Q_nb=0 | Match=1
Time=40 | clk=0 d=0 | Q_blk=0 Q_nb=0 | Match=1
Time=45 | clk=0 d=1 | Q_blk=0 Q_nb=0 | Match=1
Time=50 | clk=1 d=1 | Q_blk=1 Q_nb=1 | Match=1
Time=55 | clk=1 d=0 | Q_blk=0 Q_nb=0 | Match=1
Time=60 | clk=0 d=0 | Q_blk=0 Q_nb=0 | Match=1
Time=65 | clk=0 d=1 | Q_blk=0 Q_nb=0 | Match=1
Time=70 | clk=1 d=1 | Q_blk=1 Q_nb=1 | Match=1
Time=75 | clk=1 d=0 | Q_blk=0 Q_nb=0 | Match=1
Time=80 | clk=0 d=0 | Q_blk=0 Q_nb=0 | Match=1
Time=85 | clk=0 d=1 | Q_blk=0 Q_nb=0 | Match=1
Time=90 | clk=1 d=1 | Q_blk=1 Q_nb=1 | Match=1
Time=95 | clk=1 d=0 | Q_blk=0 Q_nb=0 | Match=1
Time=100 | clk=0 d=0 | Q_blk=0 Q_nb=0 | Match=1
tb_cd.v:35: $finish called at 110000 (1ps)
Time=110 | clk=1 d=0 | Q_blk=0 Q_nb=0 | Match=1
```

Figure 17.1: Console output of the blocking and non-blocking D latch simulation (Icarus Verilog)

17.6 RTL Schematic

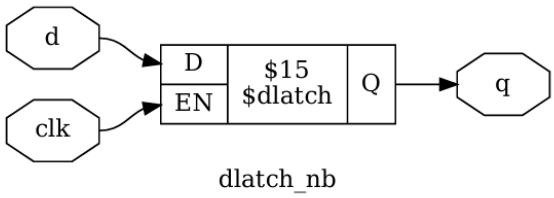


Figure 17.2: RTL schematic of the D latch generated by Yosys

17.7 Waveform

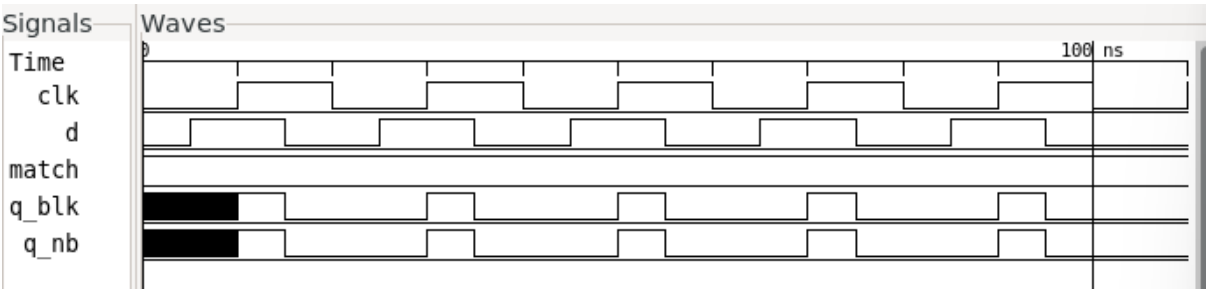


Figure 17.3: GTKWave waveform of the blocking and non-blocking D latch

17.8 Observation Table

Table 17.3: Observed Behavior of the Blocking and Non-Blocking D Latch

Time (ns)	`clk`	`d`	Q_blk	Q_nb	Match	Remark
0	0	0	x	x	1	Latch not yet written
5	0	1	x	x	1	clk low, d ignored
10	1	1	1	1	1	Transparent, Q follows d
15	1	0	0	0	1	Transparent, Q follows d
20	0	0	0	0	1	Latched, Q holds 0
25	0	1	0	0	1	Latched, d change ignored
30	1	1	1	1	1	Transparent, Q follows d
35	1	0	0	0	1	Transparent, Q follows d
40	0	0	0	0	1	Latched, Q holds 0
45	0	1	0	0	1	Latched, d change ignored
50	1	1	1	1	1	Transparent, Q follows d
55	1	0	0	0	1	Transparent, Q follows d
60	0	0	0	0	1	Latched, Q holds 0

The same pattern repeats for the remaining clock cycles up to 100 ns.

17.9 Result

The D latch was successfully simulated using both blocking and non-blocking assignments. `Q_blk` and `Q_nb` matched throughout the simulation, with `Match = 1`.

17.10 Conclusion

Both implementations produced identical latch behavior. Blocking assignments update immediately, while non-blocking assignments update at the end of the time step. Non-blocking assignments are preferred for sequential elements such as latches and flip-flops. Yosys successfully inferred a D-latch controlled by `clk`.

Experiment 18
Hamming Weight Calculation

18.1 Aim

To design and implement a circuit that calculates the Hamming weight, i.e. the number of logic 1s, of an 8-bit input.

18.2 Apparatus

OpenLane environment (Ubuntu on WSL), Icarus Verilog (`iverilog`, `vvp`), GTKWave, Yosys and Graphviz (`dot`).

18.3 Theory

The Hamming weight of a binary word is the number of bits that are 1. For an 8-bit input the weight lies between 0 (all zeros) and 8 (all ones), so a 4-bit output is required. It is used in error-correcting codes, parity generation and population-count (popcount) operations.

In this design a `for` loop inside an `always @(*)` block adds the bits of the input one by one:

$$\text{count} = \sum_{i=0}^{7} \text{data}[i]$$

The loop is unrolled by the synthesis tool into a chain of eight adders, one for each input bit, as seen in the RTL schematic. Since the block is sensitive to the whole input and `count` is assigned on every execution, the circuit is purely combinational.

18.4 Directory and Commands

Listing 18.1: Design directory and commands

```
cd ~/OpenLaneUser/designs/example
ls
hamming_weight.v  tb_hamming.v

iverilog -o hamming_sim hamming_weight.v tb_hamming.v
vvp hamming_sim
gtkwave hamming_weight.vcd
yosys -p "read_verilog hamming_weight.v; prep -top hamming_weight;
    write_dot hamming_weight_beh.dot"
dot -Tpdf hamming_weight_beh.dot -o hamming_weight_beh.pdf
explorer.exe hamming_weight_beh.pdf
```

18.5 Verilog Code

Listing 18.2: Design: Hamming Weight

```
module hamming_weight (
 input  wire [7:0] data,
 output reg  [3:0] count
);

integer i;

always @(*) begin
    count = 0;

    for (i = 0; i < 8; i = i + 1) begin
        count = count + data[i];
    end
end

endmodule
```

Listing 18.3: Testbench: Hamming Weight

```
`timescale 1ns/1ps

module hamming_weight_tb;
reg [7:0] data;
wire [3:0] count;
hamming_weight uut (data,count);

initial begin
    $dumpfile("hamming_weight.vcd");
    $dumpvars(0, hamming_weight_tb);

    $monitor("Time=%0t | Data=%b | Hamming Weight=%d",$time, data,
       count);
    data = 8'b00000000;
    #10;
    data = 8'b00000001;
    #10;
    data = 8'b10101010;
    #10;
    data = 8'b11110000;
    #10;
    data = 8'b10110101;
    #10;
    data = 8'b11111111;
    #10;
    $finish;
end
endmodule
```

18.6 Simulation Output

```
sachin@Sachin-PC:~/OpenLaneUser/designs/example$ vvp hamming_sim
VCD info: dumpfile hamming_weight.vcd opened for output.
Time=0 | Data=00000000 | Hamming Weight= 0
Time=10000 | Data=00000001 | Hamming Weight= 1
Time=20000 | Data=10101010 | Hamming Weight= 4
Time=30000 | Data=11110000 | Hamming Weight= 4
Time=40000 | Data=10110101 | Hamming Weight= 5
Time=50000 | Data=11111111 | Hamming Weight= 8
tb_hamming.v:39: $finish called at 60000 (1ps)
sachin@Sachin-PC:~/OpenLaneUser/designs/example$ gtkwave hamming_weight.vcd
```

Figure 18.1: Console output of the Hamming weight simulation (Icarus Verilog)

18.7 RTL Schematic

```
sachin@Sachin-PC:~/OpenLaneUser/designs/example$ explorer.exe hamming_weight_beh.pdf
sachin@Sachin-PC:~/OpenLaneUser/designs/example$ ls
counter       hamming_weight          leading_zero.v  tb_example.v
counter.v     hamming_weight.v        lzd.vcd         tb_hamming.v
example.v     hamming_weight.vcd      lzd_beh.dot     tb_leading.v
example.vcd   hamming_weight_beh.dot  lzd_beh.pdf
example_sim   hamming_weight_beh.pdf  lzd_sim
hamming_sim   leading_zero            tb_counter.v
sachin@Sachin-PC:~/OpenLaneUser/designs/example$
```

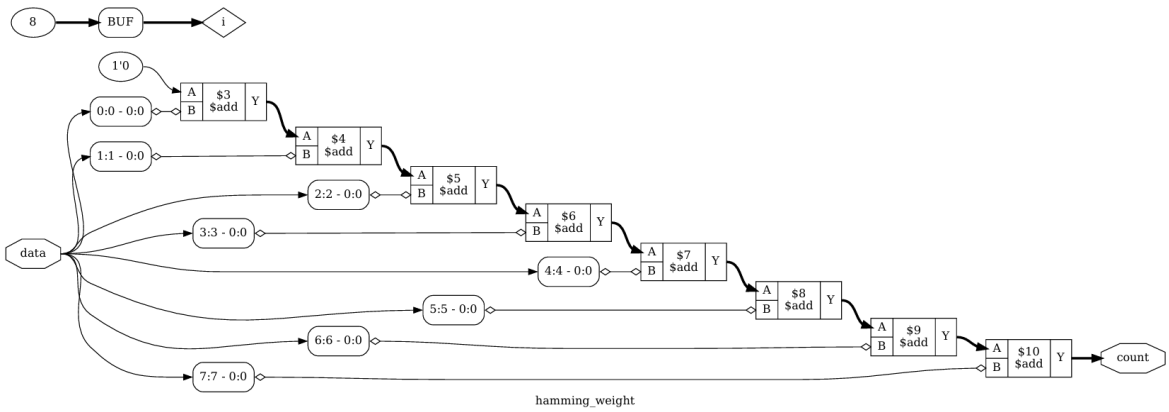


Figure 18.2: RTL schematic of the Hamming weight circuit generated by Yosys

18.8 Waveform

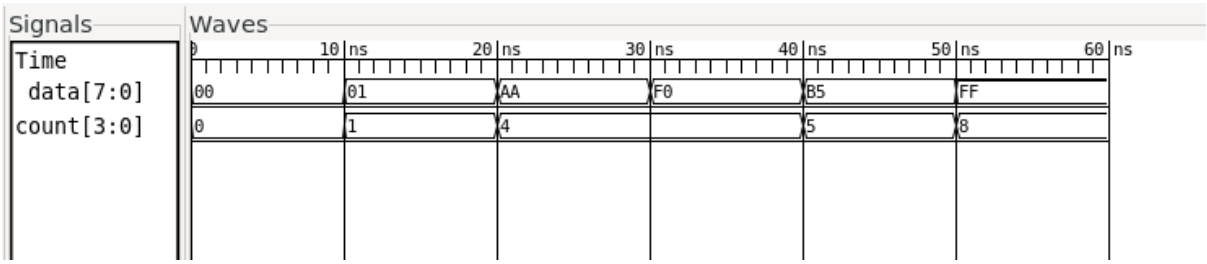


Figure 18.3: GTKWave waveform of the Hamming weight circuit

18.9 Observation Table

Table 18.1: Observed Output of the Hamming Weight Circuit

Time (ns)	Data (binary)	Data (hex)	Hamming Weight
0	00000000	00	0
10	00000001	01	1
20	10101010	AA	4
30	11110000	F0	4
40	10110101	B5	5
50	11111111	FF	8

18.10 Result

The Hamming weight circuit was simulated successfully. For the six input patterns applied, the output `count` equalled the number of 1s in `data`, from 0 for all zeros to 8 for all ones.

18.11 Conclusion

The Hamming weight of an 8-bit input was calculated by adding its bits inside a combinational `for` loop. The simulation matched the expected counts, and Yosys implemented the loop as a chain of eight adders. The 4-bit output is sufficient because the maximum weight is 8.

Experiment 19
Leading-Zero Detector

19.1 Aim

To design and implement a leading-zero detector that scans an 8-bit input starting from the MSB and stops counting as soon as the first logic 1 is encountered.

19.2 Apparatus

OpenLane environment (Ubuntu on WSL), Icarus Verilog (`iverilog`, `vvp`), GTKWave, Yosys and Graphviz (`dot`).

19.3 Theory

A leading-zero detector counts the number of consecutive 0s at the most significant end of a binary word. The count stops at the first 1. If the input is all zeros, every bit is a leading zero and the count equals the word width (8). The circuit is used in floating-point normalization and in priority encoders.

In the design a `for` loop scans the bits from bit 7 down to bit 0. While no 1 has been found, each 0 increments `count`; the first 1 sets the flag `found`, after which `count` is no longer changed. Both `count` and `found` are assigned at the start of the `always @(*)` block, so the circuit is purely combinational and no latch is inferred.

Table 19.1: Expected Output of the Leading-Zero Detector

Input (data)	Leading zeros
00000000	8
00000001	7
00000100	5
00010000	3
01000000	1
10000000	0
11111111	0

19.4 Verilog Code

Listing 19.1: Design: Leading-Zero Detector

```
1 module leading_zero_detector (
2     input  wire [7:0] data,
3     output reg  [3:0] count
4 );
5
```

```
integer i;
reg found;

always @(*) begin
    count = 0;
    found = 0;

    for (i = 7; i >= 0; i = i - 1) begin
        if (!found) begin
            if (data[i] == 1'b0)
                count = count + 1;
            else
                found = 1;
        end
    end
end

endmodule
```

Listing 19.2: Testbench: Leading-Zero Detector

```
`timescale 1ns/1ps

module leading_zero_detector_tb;
reg [7:0] data;
wire [3:0] count;

leading_zero_detector uut (data,count);

initial begin
    $dumpfile("lzd.vcd");
    $dumpvars(0, leading_zero_detector_tb);

    $monitor("Time=%0t | Data=%b | Leading Zeros=%d",
             $time, data, count);

    // Test cases
    data = 8'b00000000;
    #10;
    data = 8'b00000001;
    #10;
    data = 8'b00000010;
    #10;
    data = 8'b00000100;
    #10;
    data = 8'b00010000;
    #10;
    data = 8'b00100000;
    #10;
    data = 8'b01000000;
    #10;
    data = 8'b10000000;
    #10;
    data = 8'b11111111;
    #10;
    $finish;
end
endmodule
```

19.5 Simulation Output

```
sachin@Sachin-PC:~/OpenLaneUser/designs/example$ nano tb_leading.v
sachin@Sachin-PC:~/OpenLaneUser/designs/example$ nano leading_zero.v
sachin@Sachin-PC:~/OpenLaneUser/designs/example$ vvp lzd_sim
VCD info: dumpfile lzd.vcd opened for output.
Time=0 | Data=00000000 | Leading Zeros= 8
Time=10000 | Data=00000001 | Leading Zeros= 7
Time=20000 | Data=00000010 | Leading Zeros= 6
Time=30000 | Data=00000100 | Leading Zeros= 5
Time=40000 | Data=00010000 | Leading Zeros= 3
Time=50000 | Data=00100000 | Leading Zeros= 2
Time=60000 | Data=01000000 | Leading Zeros= 1
Time=70000 | Data=10000000 | Leading Zeros= 0
Time=80000 | Data=11111111 | Leading Zeros= 0
tb_leading.v:48: $finish called at 90000 (1ps)
sachin@Sachin-PC:~/OpenLaneUser/designs/example$ gtkwave lzd.vcd
```

Figure 19.1: Console output of the leading-zero detector simulation (Icarus Verilog)

19.6 RTL Schematic

```
sachin@Sachin-PC:~/OpenLaneUser/designs/example$ dot -Tpdf lzd_beh.dot -o lzd_beh.pdf
sachin@Sachin-PC:~/OpenLaneUser/designs/example$ explorer.exe lzd_beh.pdf
```

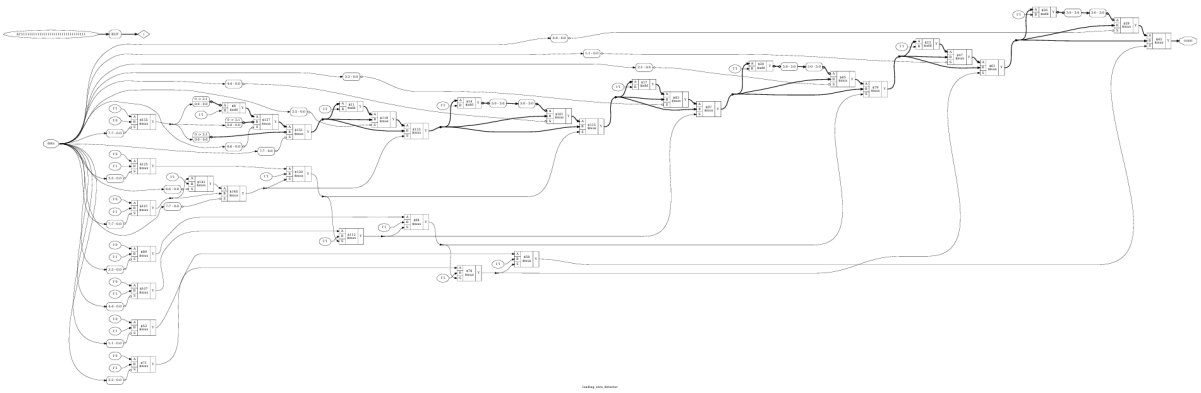


Figure 19.2: RTL schematic of the leading-zero detector generated by Yosys

19.7 Waveform

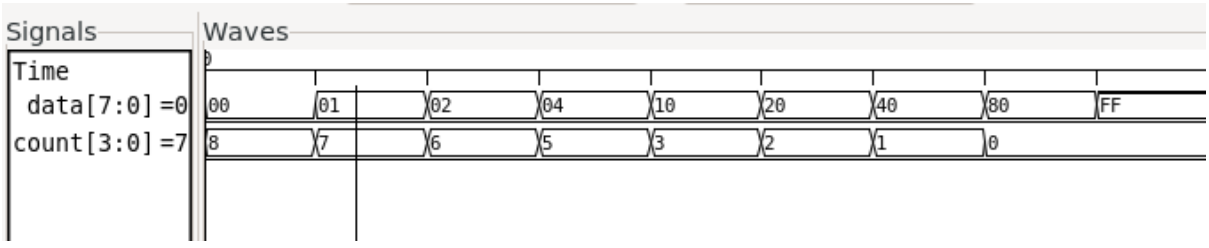


Figure 19.3: GTKWave waveform of the leading-zero detector

19.8 Observation Table

Table 19.2: Observed Output of the Leading-Zero Detector

Time (ns)	Data (binary)	Leading Zeros
0	00000000	8
10	00000001	7
20	00000010	6
30	00000100	5
40	00010000	3
50	00100000	2
60	01000000	1
70	10000000	0
80	11111111	0

19.9 Result

The leading-zero detector was simulated successfully. For every input pattern applied, the output `count` equalled the number of 0s before the first 1, from 8 for an all-zero input to 0 when the MSB is 1.

19.10 Conclusion

The detector counted the zeros from the MSB and stopped at the first 1, so bits to the right of that 1 had no effect (for example, `11111111` gives the same count as `10000000`). An all-zero input gives 8, so a 4-bit output is required. Yosys implemented the loop as a priority chain of adders and multiplexers.

Experiment 20
4-bit Up Counter with Enable and Reset

20.1 Aim

To design and implement a 4-bit up counter with enable and reset inputs. When enable is high the counter increments; when reset is high the counter is set to zero.

20.2 Apparatus

OpenLane environment (Ubuntu on WSL), Icarus Verilog (`iverilog`, `vvp`), GTKWave, Yosys and Graphviz (`dot`).

20.3 Theory

A counter is a sequential circuit whose state advances through a fixed sequence on each clock pulse. A 4-bit up counter counts from 0 to 15 (0000_2 to 1111_2) and then rolls over to 0. The design has three inputs and one output:

- `clk`: the counter updates on the rising edge of the clock.
- `reset`: when high at a clock edge, the count is set to 0000. Reset has priority over enable. Because it is checked only at the clock edge, it is a *synchronous* reset.
- `enable`: when high at a clock edge (and reset is low), the count is incremented by 1; when low, the count is held.

The count is stored in a 4-bit register, which Yosys implements as a D flip-flop bank with an adder and two multiplexers, as seen in the RTL schematic.

Table 20.1: Function Table of the Counter (at each rising clock edge)

reset	enable	Next count
1	X	0000
0	0	count (hold)
0	1	count + 1

20.4 Directory and Commands

Listing 20.1: Design directory and commands

```
cd ~/OpenLaneUser/designs/counter
ls
counter.v  tb_counter.v

```

```
iverilog -o counter_sim counter.v tb_counter.v
vvp counter_sim
gtkwave counter.vcd
yosys -p "read_verilog counter.v; prep -top counter; write_dot
   counter_beh.dot"
dot -Tpdf counter_beh.dot -o counter_beh.pdf
explorer.exe counter_beh.pdf
```

20.5 Verilog Code

Listing 20.2: Design: 4-bit Up Counter

```
module counter(
    input wire clk,
    input wire reset,
    input wire enable,

    output reg[3:0] count
);

always @(posedge clk) begin
     if (reset)
        count <= 4'b0000;
     else if(enable)
        count <=count+1'b1;

end
endmodule
```

Listing 20.3: Testbench: 4-bit Up Counter

```
`timescale 1ns/1ps
module counter_tb;
reg clk;
reg reset;
reg enable;
wire [3:0] count;

counter uut (clk,reset,enable,count);
always begin
    #5 clk = ~clk;
end

initial begin
    $dumpfile("counter.vcd");
    $dumpvars(0, counter_tb);
    $monitor("Time=%0t | Reset=%b | Enable=%b | Count=%b",
             $time, reset, enable, count);
    clk = 0;
    reset = 1;
    enable = 0;
    #10;
    reset = 0;
    enable = 1;
    #40;
    enable = 0;
```

```
    #20;
    enable = 1;
    #30;
    reset = 1;
    #10;
    reset = 0;
    enable = 1;
    #30;
    $finish;
end
endmodule
```

20.6 Simulation Output

```
sachin@Sachin-PC:~/OpenLaneUser/designs/counter$ ls
counter.v  tb_counter.v
sachin@Sachin-PC:~/OpenLaneUser/designs/counter$ nano counter.v
sachin@Sachin-PC:~/OpenLaneUser/designs/counter$ nano tb_counter.v
sachin@Sachin-PC:~/OpenLaneUser/designs/counter$ ls
counter.v  tb_counter.v
sachin@Sachin-PC:~/OpenLaneUser/designs/counter$ iverilog -o counter_sim counter.v tb_counter.v
sachin@Sachin-PC:~/OpenLaneUser/designs/counter$ vvp counter_sim
VCD info: dumpfile counter.vcd opened for output.
Time=0 | Reset=1 | Enable=0 | Count=xxxx
Time=5000 | Reset=1 | Enable=0 | Count=0000
Time=10000 | Reset=0 | Enable=1 | Count=0000
Time=15000 | Reset=0 | Enable=1 | Count=0001
Time=25000 | Reset=0 | Enable=1 | Count=0010
Time=35000 | Reset=0 | Enable=1 | Count=0011
Time=45000 | Reset=0 | Enable=1 | Count=0100
Time=50000 | Reset=0 | Enable=0 | Count=0100
Time=70000 | Reset=0 | Enable=1 | Count=0100
Time=75000 | Reset=0 | Enable=1 | Count=0101
Time=85000 | Reset=0 | Enable=1 | Count=0110
Time=95000 | Reset=0 | Enable=1 | Count=0111
Time=100000 | Reset=1 | Enable=1 | Count=0111
Time=105000 | Reset=1 | Enable=1 | Count=0000
Time=110000 | Reset=0 | Enable=1 | Count=0000
Time=115000 | Reset=0 | Enable=1 | Count=0001
Time=125000 | Reset=0 | Enable=1 | Count=0010
Time=135000 | Reset=0 | Enable=1 | Count=0011
tb_counter.v:34: $finish called at 140000 (1ps)
sachin@Sachin-PC:~/OpenLaneUser/designs/counter$ gtkwave counter.vcd

GTKWave Analyzer v3.3.116 (w)1999-2023 BSI

[0] start time.
[140000] end time.
WM Destroy
sachin@Sachin-PC:~/OpenLaneUser/designs/counter$ yosys -p "read_verilog counter.v; prep -top cou
```

Figure 20.1: Console output of the counter simulation (Icarus Verilog)

20.7 RTL Schematic

```
sachin@Sachin-PC:~/OpenLaneUser/designs/counter$ dot -Tpdf counter_beh.dot -o counter_beh.pdf
sachin@Sachin-PC:~/OpenLaneUser/designs/counter$ explorer.exe counter_beh.pdf
```

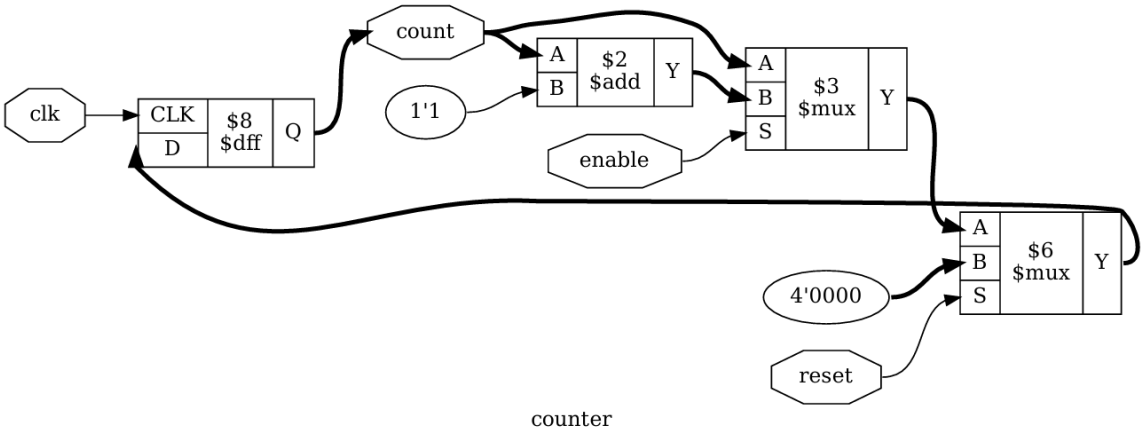


Figure 20.2: RTL schematic of the 4-bit counter generated by Yosys

20.8 Waveform

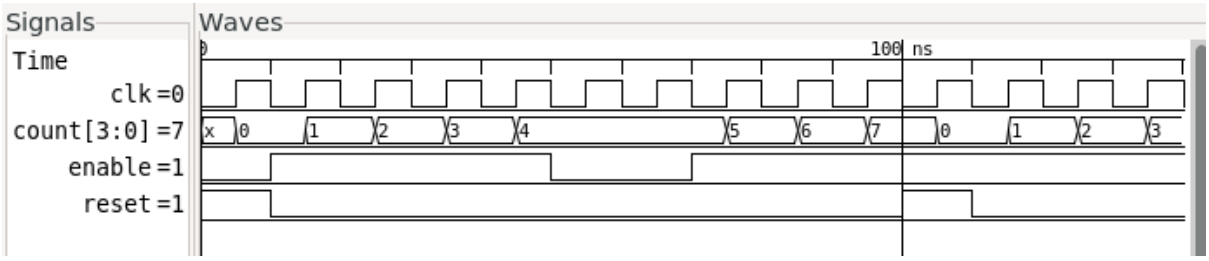


Figure 20.3: GTKWave waveform of the 4-bit counter

20.9 Observation Table

Table 20.2: Observed Output of the 4-bit Counter (Selected Clock Edges)

Time (ns)	reset	enable	count	Remark
5	1	0	0000	Reset clears the count
15	0	1	0001	Counting starts
45	0	1	0100	Count reaches 4
50–70	0	0	0100	Enable low: count held
75	0	1	0101	Counting resumes
95	0	1	0111	Count reaches 7
105	1	1	0000	Reset has priority over enable
115	0	1	0001	Counting restarts from 0

20.10 Result

The 4-bit up counter was simulated successfully. The count incremented on each rising clock edge while enable was high, held its value while enable was low, and returned to zero when reset was applied.

20.11 Conclusion

The counter behaved as specified: reset (synchronous, with priority over enable) cleared the count to 0000, enable controlled whether the count incremented, and the count then continued from zero after reset was released. Yosys implemented the design as a register with an incrementer and two multiplexers for enable and reset.

Experiment 21
Sequence Detector for 101 with Overlapping Detection

21.1 Aim

To design and implement a sequence detector that detects the sequence 101 in a serial input stream. The output goes high as soon as the sequence 101 is detected, and overlapping detection is allowed.

21.2 Apparatus

OpenLane environment (Ubuntu on WSL), Icarus Verilog (`iverilog`, `vvp`), GTKWave, Yosys and Graphviz (`dot`).

21.3 Theory

A sequence detector is an FSM that detects a specific bit pattern in a serial input. In overlapping detection, part of a detected pattern can also start the next pattern, such as two occurrences of `101` in `10101`. The detector is a Moore machine with four states:

The FSM uses a state register, next-state logic, and output logic. Transitions from `S_101` preserve possible overlapping patterns, enabling continuous detection.

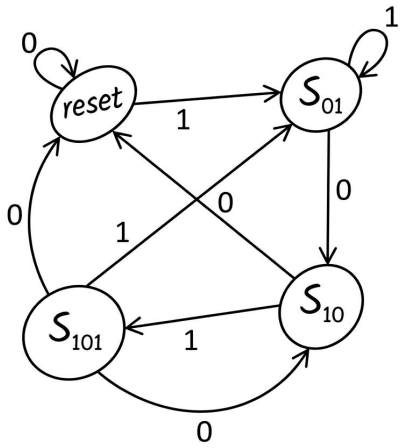


Figure 21.1: FSM

21.4 Verilog Code

Listing 21.1: Design: 101 Sequence Detector (Overlapping)

```
module sd(

    input wire clk ,rst,d,
    output reg out
);
parameter S_res = 2'b00;
parameter S_1   = 2'b01;
parameter S_10  = 2'b10;
parameter S_101 = 2'b11;

reg [1:0] curr, next;

always @(posedge clk) begin

    if(rst) begin
      curr <= 2'b00;
    end else begin
      curr <= next ;
    end

end


always @(*) begin
  case(curr)
     S_res : next= d? S_1 :S_res;
     S_1   : next= d? S_1 :S_10;
     S_10  : next= d? S_101 :S_res;
     S_101 : next= d? S_1 :S_10;
     default:next=S_res;
  endcase
end

always @(*)begin
  if (curr ==S_101) begin
    out = 1'b1;
  end else begin
    out = 1'b0;
  end
end

endmodule
```

Listing 21.2: Testbench: 101 Sequence Detector

```
`timescale 1ns/1ps


module sd_tb();
reg clk,rst,d;
wire out;

sd uut(clk, rst, d, out);

```

```
always #5 clk =~clk;
initial begin
    $dumpfile("sd.vcd");
    $dumpvars(1, sd_tb);

    $monitor("time=%0t | rst=%b | d=%b | out=%b",$time, rst, d, out);
    clk = 0;
    rst = 1;
    d = 0;

    @(negedge clk);
    rst = 0;

    // Send 101
    @(negedge clk); d = 1;
    @(negedge clk); d = 0;
    @(negedge clk); d = 1;

    // Send another 101
    @(negedge clk); d = 0;
    @(negedge clk); d = 1;
    @(negedge clk); d = 0;
    @(negedge clk); d = 1;

    #10;
    $finish;
end
endmodule
```

21.5 Simulation Output

```
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ iverilog -o sd_sim sd.v tb_sd.v
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ ls
led.v  sd  sd.v  sd_sim  sequence_101.v  tb_sd.v  vm.v
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ vvp sd_sim
VCD info: dumpfile sd.vcd opened for output.
time=0 | rst=1 | d=0 | out=x
time=5000 | rst=1 | d=0 | out=0
time=10000 | rst=0 | d=0 | out=0
time=20000 | rst=0 | d=1 | out=0
time=30000 | rst=0 | d=0 | out=0
time=40000 | rst=0 | d=1 | out=0
time=45000 | rst=0 | d=1 | out=1
time=50000 | rst=0 | d=0 | out=1
time=55000 | rst=0 | d=0 | out=0
time=60000 | rst=0 | d=1 | out=0
time=65000 | rst=0 | d=1 | out=1
time=70000 | rst=0 | d=0 | out=1
time=75000 | rst=0 | d=0 | out=0
time=80000 | rst=0 | d=1 | out=0
time=85000 | rst=0 | d=1 | out=1
tb_sd.v:34: $finish called at 90000 (1ps)
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ gtkwave mux.vcd

GTKWave Analyzer v3.3.116 (w)1999-2023 BSI
```

Figure 21.2: Console output of the 101 detector simulation (Icarus Verilog)

21.6 RTL Schematic

```
Yosys 0.33 (git sha1 2584903a060)
Time spent: 22% 6x opt_clean (0 sec), 22% 7x opt_expr (0 sec), ...
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ dot -V
dot - graphviz version 2.43.0 (0)
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ dot -Tpdf sd_beh.dot -o sd_beh.pdf
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ ls -lh sd_beh.pdf
-rw-r--r-- 1 sachin sachin 21K Oct  3 18:04 sd_beh.pdf
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ explorer.exe sd_beh.pdf
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ ls
led.v  sd.v    sd_beh.dot  sd_sim          tb_sd.v
sd     sd.vcd  sd_beh.pdf  sequence_101.v  vm.v
```

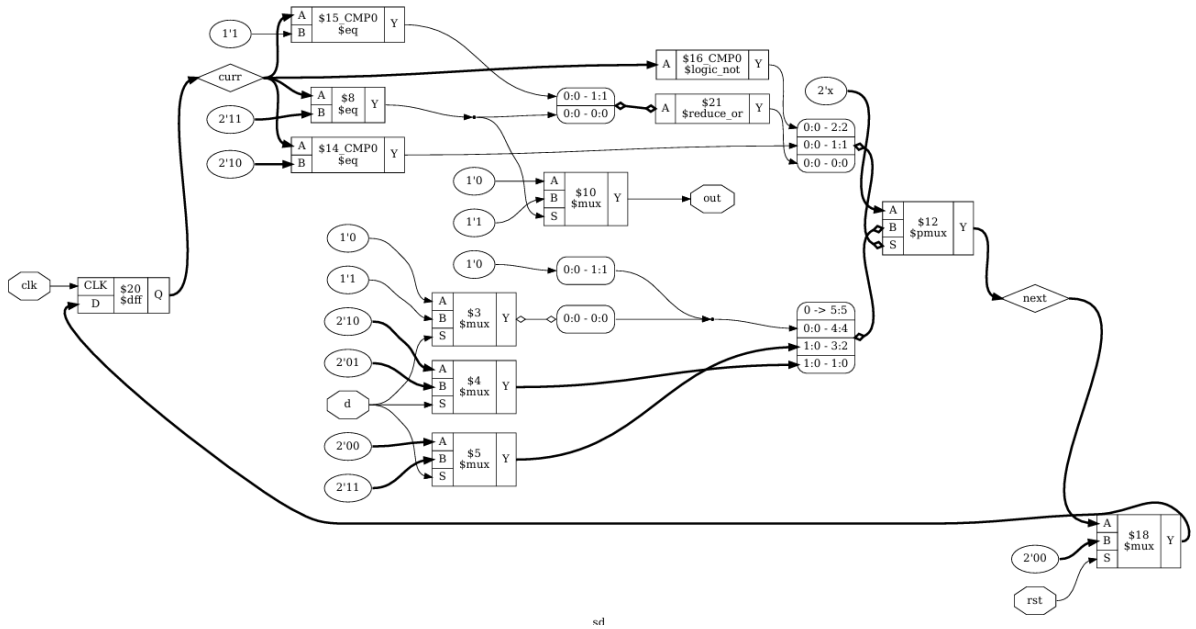


Figure 21.3: RTL schematic of the 101 detector generated by Yosys

21.7 Waveform

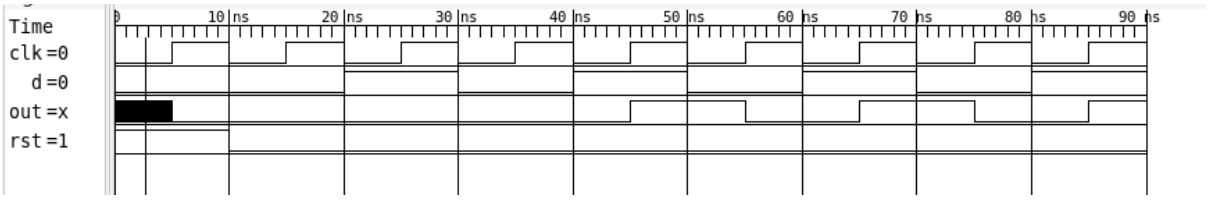


Figure 21.4: GTKWave waveform of the 101 detector

21.8 Observation Table

The testbench applies the bit stream 1010101 (one bit per clock, changing on the falling edge). This stream contains three overlapping occurrences of 101.

Table 21.1: Observed Output of the 101 Detector

Time (ns)	d	out	Remark
5–10	0	0	Reset applied, then released
20	1	0	First bit (1) applied
30	0	0	Second bit (0) applied
40	1	0	Third bit (1) applied
45–50	1, 0	1	First 101 detected after the clock edge
60	1	0	Next bit applied (the earlier 1 is reused)
65–70	1, 0	1	Second 101 detected (overlapping)
80	1	0	Next bit applied
85	1	1	Third 101 detected (overlapping)

21.9 Result

The 101 sequence detector was simulated successfully. The output went high three times for the input stream 1010101, once for each overlapping occurrence of 101, and stayed high for one clock period each time.

21.10 Conclusion

The Moore FSM with four states detected the pattern 101 and allowed overlapping occurrences, because from the detection state a 1 returns to `S_1` and a 0 returns to `S_10`. Because the output is decoded from the state, it appears one clock edge after the last bit of the sequence. Yosys implemented the design as a 2-bit state register with a synchronous reset and multiplexer-based next-state logic.

Experiment 22
Implementation of a Vending Machine Controller

22.1 Aim

To design and implement a vending machine controller that accepts Rs. 5 and Rs. 10 coins, keeps track of the amount received, and activates its output when the accumulated amount reaches Rs. 15. After Rs. 15 has been reached the controller returns to the initial state.

22.2 Apparatus

OpenLane environment (Ubuntu on WSL), Icarus Verilog (`iverilog`, `vvp`), GTKWave, Yosys and Graphviz (`dot`).

22.3 Theory

The controller is a finite state machine whose state is the amount collected so far. It has two coin inputs, `d_5` (Rs. 5 coin) and `d_10` (Rs. 10 coin), a clock `clk`, and an asynchronous active-low reset `rst`. The four states are:

- `idle`: no money received (Rs. 0),R_5: Rs. 5 received,R_10: Rs. 10 received,R_15: Rs. 15 (or more) received.

If both coin inputs are high at the same time, `d_5` has priority. Any total of Rs. 15 or more is treated as Rs. 15. From `R_15` the controller returns to `idle` at the next clock edge.The output `out` is decoded from the next state: it is 1 whenever the next state is `R_15`.

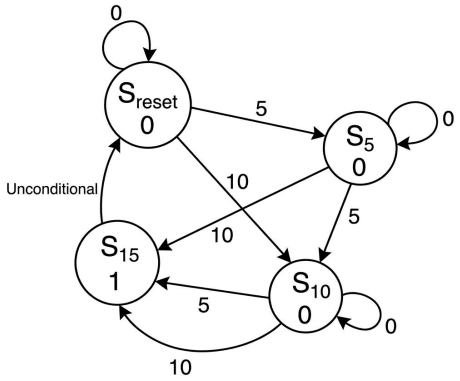


Figure 22.1: FSM

22.4 Verilog Code

Listing 22.1: Design: Vending Machine Controller

```
module vm(

   input wire clk,rst,d_5,d_10,
   output reg out
);
reg [1:0] curr,next;

 parameter idle =2'b00;
parameter R_5  =2'b01;
parameter R_10 =2'b10;
parameter R_15 =2'b11;
always @(posedge clk or negedge rst) begin  //asynchronous active low
   if(!rst) begin
     curr <=idle;
   end else begin
     curr <= next;
   end
end
always @(*) begin
    case(curr)
       idle: next= d_5 ? R_5  :(d_10 ? R_10 :idle);
       R_5 : next= d_5 ? R_10 :(d_10 ? R_15 :R_5 );
       R_10: next= d_5 ? R_15 :(d_10 ? R_15 :R_10);
       R_15: next= idle;
    default: next = idle;
    endcase
end
always@(*) begin

  if ( next ==R_15) begin
     out =1'b1;
  end else begin
     out =1'b0;
  end
end
endmodule
```

Listing 22.2: Testbench: Vending Machine Controller

```
`timescale 1ns/1ps

module vm_tb;
    reg clk, rst, d_5, d_10;
    wire out;

    vm uut(clk, rst, d_5, d_10, out);

    always #5 clk = ~clk;

    initial begin
        $dumpfile("vm.vcd");
        $dumpvars(1, vm_tb);

```

```
        $monitor("Time = %t | Clk = %b | rst = %b | Rs5 = %b | Rs10 = %
            b | Current_State = %b | Out = %b",
                $time, clk, rst, d_5, d_10, uut.curr, out);

        clk = 0;
        rst = 0;
        d_5 = 0;
        d_10 = 0;
        #10;
        rst = 1;
        #20;

        // Rs. 5 + Rs. 5 + Rs. 5 = Rs. 15
        @(negedge clk) d_5 = 1;
        @(negedge clk) d_5 = 0;
        #10;
        @(negedge clk) d_5 = 1;
        @(negedge clk) d_5 = 0;
        #10;
        @(negedge clk) d_5 = 1;
        @(negedge clk) d_5 = 0;

        #35;

        // Rs. 10 + Rs. 10
        @(negedge clk) d_10 = 1;
        @(negedge clk) d_10 = 0;
        #10;
        @(negedge clk) d_10 = 1;
        @(negedge clk) d_10 = 0;

        #40;
        $finish;
    end
endmodule
```

22.5 Simulation Output

```
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ nano tb_vm.v
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ iverilog -o vm_sim vm.v tb_vm.v
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ vvp vm_sim
VCD info: dumpfile vm.vcd opened for output.
Time =                     0 | Clk = 0 | rst = 0 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                  5000 | Clk = 1 | rst = 0 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                 10000 | Clk = 0 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                 15000 | Clk = 1 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                 20000 | Clk = 0 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                 25000 | Clk = 1 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                 30000 | Clk = 0 | rst = 1 | Rs5 = 1 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                 35000 | Clk = 1 | rst = 1 | Rs5 = 1 | Rs10 = 0 | Current_State = 01 | Out = 0
Time =                 40000 | Clk = 0 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 01 | Out = 0
Time =                 45000 | Clk = 1 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 01 | Out = 0
Time =                 50000 | Clk = 0 | rst = 1 | Rs5 = 1 | Rs10 = 0 | Current_State = 01 | Out = 0
Time =                 55000 | Clk = 1 | rst = 1 | Rs5 = 1 | Rs10 = 0 | Current_State = 10 | Out = 1
Time =                 60000 | Clk = 0 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 10 | Out = 0
Time =                 65000 | Clk = 1 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 10 | Out = 0
Time =                 70000 | Clk = 0 | rst = 1 | Rs5 = 1 | Rs10 = 0 | Current_State = 10 | Out = 1
Time =                 75000 | Clk = 1 | rst = 1 | Rs5 = 1 | Rs10 = 0 | Current_State = 11 | Out = 0
Time =                 80000 | Clk = 0 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 11 | Out = 0
Time =                 85000 | Clk = 1 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                 90000 | Clk = 0 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                 95000 | Clk = 1 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                100000 | Clk = 0 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                105000 | Clk = 1 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                110000 | Clk = 0 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                115000 | Clk = 1 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                120000 | Clk = 0 | rst = 1 | Rs5 = 0 | Rs10 = 1 | Current_State = 00 | Out = 0
Time =                125000 | Clk = 1 | rst = 1 | Rs5 = 0 | Rs10 = 1 | Current_State = 10 | Out = 1
Time =                130000 | Clk = 0 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 10 | Out = 0
Time =                135000 | Clk = 1 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 10 | Out = 0
Time =                140000 | Clk = 0 | rst = 1 | Rs5 = 0 | Rs10 = 1 | Current_State = 10 | Out = 1
Time =                145000 | Clk = 1 | rst = 1 | Rs5 = 0 | Rs10 = 1 | Current_State = 11 | Out = 0
Time =                150000 | Clk = 0 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 11 | Out = 0
Time =                155000 | Clk = 1 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                160000 | Clk = 0 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                165000 | Clk = 1 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                170000 | Clk = 0 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                175000 | Clk = 1 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
Time =                180000 | Clk = 0 | rst = 1 | Rs5 = 0 | Rs10 = 0 | Current_State = 00 | Out = 0
```

Figure 22.2: Console output of the vending machine simulation (Icarus Verilog)

22.6 RTL Schematic

```
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ dot -Tpdf vm_beh.dot -o vm_beh.pdf
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ ls
led.v  sd  sd.v  sd.vcd  sd_beh.dot  sd_beh.pdf  sd_sim  sequence_101.v  tb_sd.v  tb_vm.v  vm.v  vm.vcd  vm_beh.dot  vm_beh.pdf  vm_sim
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ explorer.exe vm_beh.pdf
```

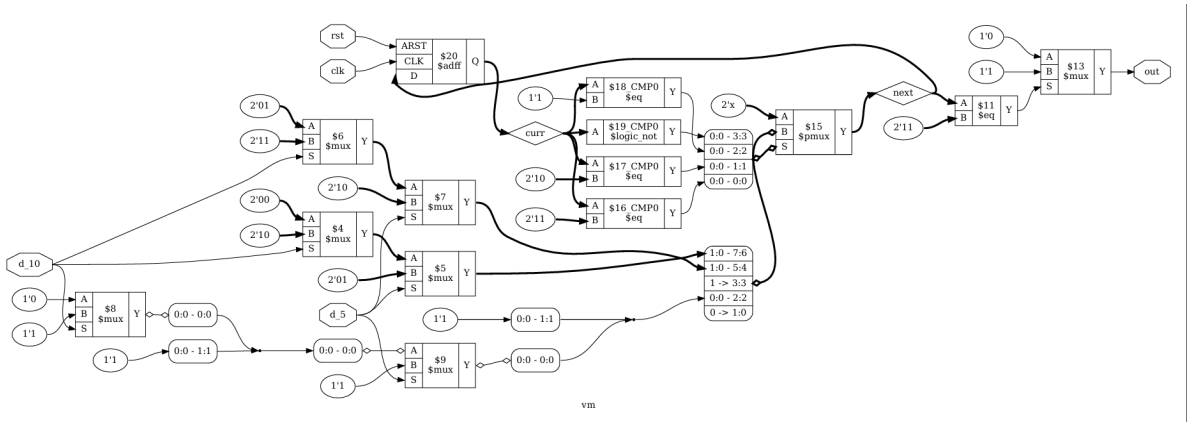


Figure 22.3: RTL schematic of the vending machine controller generated by Yosys

22.7 Waveform

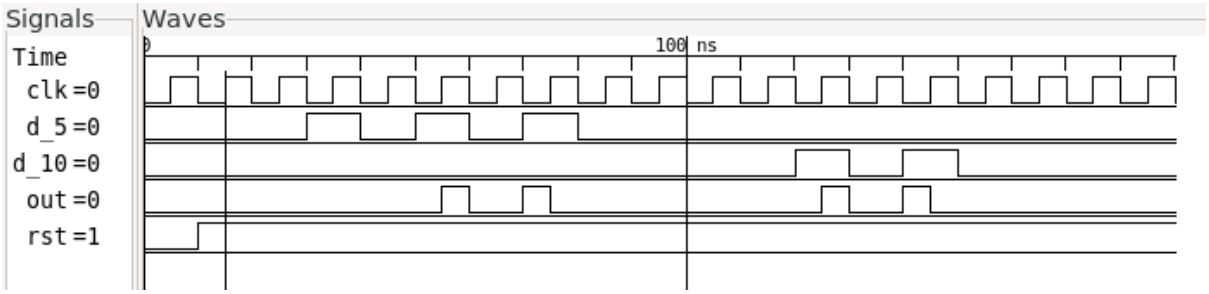


Figure 22.4: GTKWave waveform of the vending machine controller

22.8 Observation Table

Table 22.1: Observed Behavior of the Vending Machine Controller

Time (ns)	d_5	d_10	State	out	Remark
10	0	0	00	0	Reset released
35	1	0	01	0	First Rs. 5 coin: R_5
55	1	0	10	1	Second Rs. 5 coin: R_10; coin still high, so next = R_15
70	1	0	10	1	Third Rs. 5 coin applied: next = R_15
75	1	0	11	0	R_15 reached (Rs. 15)
85	0	0	00	0	Back to idle
125	0	1	10	1	First Rs. 10 coin: R_10
140	0	1	10	1	Second Rs. 10 coin applied: next = R_15
145	0	1	11	0	R_15 reached
155	0	0	00	0	Back to idle

22.9 Result

The vending machine controller was simulated successfully. Three Rs. 5 coins and two Rs. 10 coins each took the state machine to `R_15` (state 11), and the controller returned to the `idle` state at the following clock edge.

22.10 Conclusion

The FSM tracked the amount collected through the states idle, R_5, R_10 and R_15. The output `out` follows the next state, so it is high when the coin that makes the total Rs. 15 is being inserted; in this testbench it also goes high for 5 ns after the second Rs. 5 coin, because the coin input is still high just after the clock edge. After reaching Rs. 15 the controller always returned to idle, ready for the next customer. Yosys implemented the design as a state register with asynchronous reset, comparators and multiplexers for the next-state logic.

Experiment 23
Implementation of a Rising-Edge Detector with Digital Switch

23.1 Aim

To design and implement a rising-edge detector using a digital switch (push button). The output goes high when a $0 \rightarrow 1$ transition occurs on the button input and returns low at the next clock edge, even if the button remains pressed. The design is implemented using a three-block FSM.

23.2 Apparatus

OpenLane environment (Ubuntu on WSL), Icarus Verilog (`iverilog`, `vvp`), GTKWave, Yosys and Graphviz (`dot`).

23.3 Theory

A rising-edge detector generates a short pulse when the input changes from 0 to 1, ensuring a held button is counted only once. The FSM uses two states, `low` and `high`, and follows a three-block style:

1. **State register:** Updates the state on the rising clock edge with asynchronous active-low reset.
2. **Next-state logic:** The next state follows the button input `b`.
3. **Output logic:** Output is high when the state is `low` and `b`=1.

After the next clock edge, the state becomes `high`, making the output low while the button remains pressed.

23.4 Verilog Code

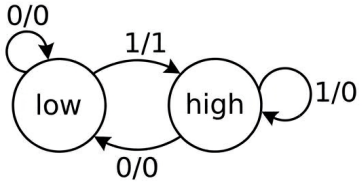


Figure 23.1: FSM

Listing 23.1: Design: Rising-Edge Detector (Three-Block FSM)

```
module button(
    input wire clk, rst, b,
    output reg out
);

parameter low  = 1'b0;
parameter high = 1'b1;

reg current, next;

always @(posedge clk or negedge rst) begin
    if(!rst) begin
        current <= low;
    end else begin
        current <= next;
    end
end

always @(*) begin
    case(current)
        low : next = b ? high : low;
        high: next = b ? high : low;
        default: next = low;
    endcase
end

always @(*) begin
    if(current == 0 && b == 1) begin
        out = 1'b1;
    end else begin
        out = 1'b0;
    end
end

endmodule
```

Listing 23.2: Testbench: Rising-Edge Detector

```
`timescale 1ns/1ps

module button_tb(
);
    reg clk, rst, b;
    wire out;
    button utt(clk, rst, b, out);

    always #5 clk = ~clk;

    initial begin
        $dumpfile("button.vcd");
        $dumpvars(1, button_tb);
        $monitor("Time = %t | Clk = %b | Reset = %b | B = %b | Out = %b
            ", $time, clk, rst, b, out);
        clk = 0;
        b = 0;
        rst = 0;
        #10;
```

```
        rst = 1;
        #20;
        @(negedge clk) b = 1;
        @(negedge clk) b = 0;
        #40;
        @(negedge clk) b = 1;
        #80;
        @(negedge clk) b = 0;
        #40;
        $finish;
    end
endmodule
```

23.5 Simulation Output

```
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ iverilog -o led_sim led.v tb_led.
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ vvp led_sim
VCD info: dumpfile button.vcd opened for output.
Time =                     0 | Clk = 0 | Reset = 0 | B = 0 | Out = 0
Time =                  5000 | Clk = 1 | Reset = 0 | B = 0 | Out = 0
Time =                 10000 | Clk = 0 | Reset = 1 | B = 0 | Out = 0
Time =                 15000 | Clk = 1 | Reset = 1 | B = 0 | Out = 0
Time =                 20000 | Clk = 0 | Reset = 1 | B = 0 | Out = 0
Time =                 25000 | Clk = 1 | Reset = 1 | B = 0 | Out = 0
Time =                 30000 | Clk = 0 | Reset = 1 | B = 1 | Out = 1
Time =                 35000 | Clk = 1 | Reset = 1 | B = 1 | Out = 0
Time =                 40000 | Clk = 0 | Reset = 1 | B = 0 | Out = 0
Time =                 45000 | Clk = 1 | Reset = 1 | B = 0 | Out = 0
Time =                 50000 | Clk = 0 | Reset = 1 | B = 0 | Out = 0
Time =                 55000 | Clk = 1 | Reset = 1 | B = 0 | Out = 0
Time =                 60000 | Clk = 0 | Reset = 1 | B = 0 | Out = 0
Time =                 65000 | Clk = 1 | Reset = 1 | B = 0 | Out = 0
Time =                 70000 | Clk = 0 | Reset = 1 | B = 0 | Out = 0
Time =                 75000 | Clk = 1 | Reset = 1 | B = 0 | Out = 0
Time =                 80000 | Clk = 0 | Reset = 1 | B = 1 | Out = 1
Time =                 85000 | Clk = 1 | Reset = 1 | B = 1 | Out = 0
Time =                 90000 | Clk = 0 | Reset = 1 | B = 1 | Out = 0
Time =                 95000 | Clk = 1 | Reset = 1 | B = 1 | Out = 0
Time =                100000 | Clk = 0 | Reset = 1 | B = 1 | Out = 0
Time =                105000 | Clk = 1 | Reset = 1 | B = 1 | Out = 0
Time =                110000 | Clk = 0 | Reset = 1 | B = 1 | Out = 0
Time =                115000 | Clk = 1 | Reset = 1 | B = 1 | Out = 0
Time =                120000 | Clk = 0 | Reset = 1 | B = 1 | Out = 0
Time =                125000 | Clk = 1 | Reset = 1 | B = 1 | Out = 0
Time =                130000 | Clk = 0 | Reset = 1 | B = 1 | Out = 0
Time =                135000 | Clk = 1 | Reset = 1 | B = 1 | Out = 0
Time =                140000 | Clk = 0 | Reset = 1 | B = 1 | Out = 0
Time =                145000 | Clk = 1 | Reset = 1 | B = 1 | Out = 0
Time =                150000 | Clk = 0 | Reset = 1 | B = 1 | Out = 0
Time =                155000 | Clk = 1 | Reset = 1 | B = 1 | Out = 0
Time =                160000 | Clk = 0 | Reset = 1 | B = 0 | Out = 0
Time =                165000 | Clk = 1 | Reset = 1 | B = 0 | Out = 0
Time =                170000 | Clk = 0 | Reset = 1 | B = 0 | Out = 0
Time =                175000 | Clk = 1 | Reset = 1 | B = 0 | Out = 0
Time =                180000 | Clk = 0 | Reset = 1 | B = 0 | Out = 0
Time =                185000 | Clk = 1 | Reset = 1 | B = 0 | Out = 0
```

Figure 23.2: Console output of the rising-edge detector simulation (Icarus Verilog)

23.6 RTL Schematic

```
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ dot -Tpdf button_beh.dot -o button_beh.pdf
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ explorer.exe button_beh.pdf
```

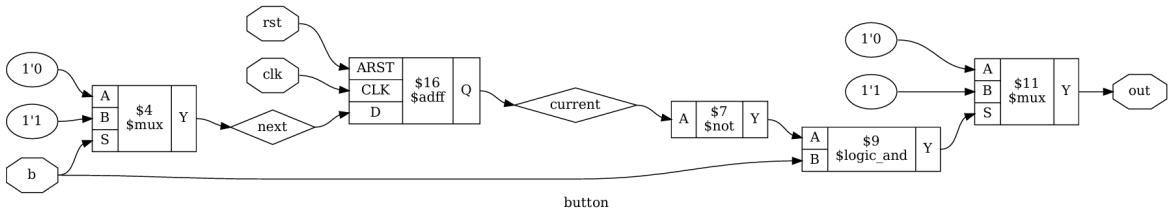


Figure 23.3: RTL schematic of the rising-edge detector generated by Yosys

23.7 Waveform

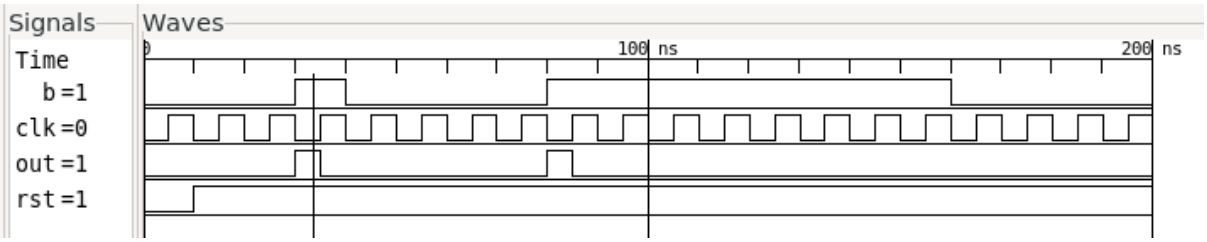


Figure 23.4: GTKWave waveform of the rising-edge detector

23.8 Observation Table

Table 23.1: Observed Output of the Rising-Edge Detector

Time (ns)	b	out	Remark
0–10	0	0	Reset applied
30	1	1	Button pressed (0 → 1): output pulse
35	1	0	Next clock edge: output returns low
40	0	0	Button released
80	1	1	Button pressed again: output pulse
85–155	1	0	Button held: no further pulses
160	0	0	Button released

23.9 Result

The rising-edge detector was simulated successfully. The output went high only when the button changed from 0 to 1, and went low again at the next clock edge even while the button was held down for 80 ns.

Experiment 24
Implementation of a Sensor-Based Traffic Signal Controller

24.1 Aim

To design and verify a sensor-based traffic signal controller using a Finite State Machine (FSM). The controller normally keeps the highway signal in the green state. When a vehicle is detected on the less-travelled road, the controller changes the highway signal from green to yellow and then to red. After the vehicle has passed, the signal changes from red to yellow and finally returns to green.

24.2 Apparatus

OpenLane environment (Ubuntu on WSL), Icarus Verilog (`iverilog`, `vvp`), GTKWave, Yosys and Graphviz (`dot`).

24.3 Theory

The traffic signal controller is implemented as a four-state Finite State Machine (FSM), with transitions controlled by the vehicle detection sensor.

When a vehicle is detected in the GREEN state, the controller changes to YELLOW and then RED. It remains RED while the vehicle is present. When the sensor becomes inactive, it passes through the second YELLOW state and returns to GREEN.

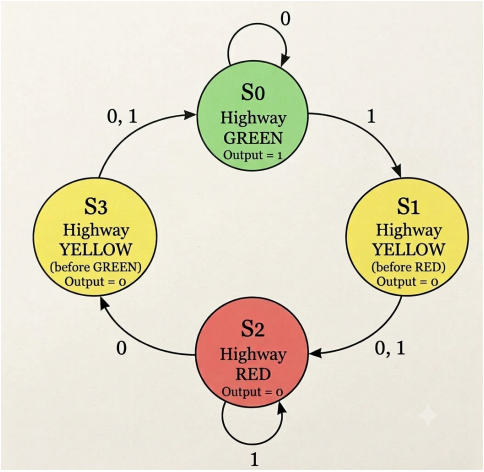


Figure 24.1: FSM

24.4 Verilog Code

Listing 24.1: Design: Sensor-Based Traffic Signal Controller

```
`timescale 1ns/1ps

module traffic (
    input  clk,rst,sensor,
    output reg y
);
    reg [1:0] current,next;

    parameter GREEN  = 2'b00;
    parameter YELLOW = 2'b01;
    parameter RED    = 2'b10;
    parameter YEL2   = 2'b11;

    always @(posedge clk) begin
        if (!rst)
            current <= GREEN;
        else
            current <= next;
    end

    always @(*) begin
        case (current)

            GREEN :next = sensor ? YELLOW : GREEN;
            YELLOW:next = RED;
            RED   :next = sensor ? RED : YEL2;
            YEL2  :next =  GREEN;
          default :next = GREEN;

        endcase
    end

    always @(*) begin
        if (current == GREEN)begin
            y = 1'b1;
       end else begin
            y = 1'b0;
       end
    end

endmodule
```

24.5 Testbench

Listing 24.2: Testbench: Traffic Signal Controller

```
`timescale 1ns/1ps

module traffic_tb;

    reg clk,rst,sensor;
```

```
    wire y;

    traffic_signal_controller uut (clk,rst,sensor,y);

    always #5 clk = ~clk;

    initial begin

        // VCD generation
        $dumpfile("traffic.vcd");
        $dumpvars(0, traffic_signal_controller_tb);

        // Initial values
        clk = 0;
        rst = 0;
        sensor = 0;

        // Reset
        #10;
        rst = 1;

        // No vehicle -> remain GREEN
        sensor = 0;
        #20;

        // Vehicle detected
        // GREEN -> YELLOW -> RED
        sensor = 1;
        #20;

        // Vehicle still present -> remain RED
        sensor = 1;
        #20;

        // Vehicle has passed
        // RED -> YELLOW -> GREEN
        sensor = 0;
        #20;

        // Remain GREEN
        sensor = 0;
        #20;

        $finish;
    end

endmodule
```

24.6 Simulation Output

```
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ nano tb_traffic.v
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$  iverilog -o traffic_sim traffic.v tb_traffic.v
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$  vvp traffic_sim
VCD info: dumpfile traffic.vcd opened for output.
tb_traffic.v:52: $finish called at 110000 (1ps)
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ gtkwave traffic.vcd

GTKWave Analyzer v3.3.116 (w)1999-2023 BSI
```

Figure 24.2: Console output of the rising-edge detector simulation (Icarus Verilog)

24.7 RTL Schematic

```
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ dot -Tpdf traffic_fsm.dot -o traffic_fsm.pdf
sachin@Sachin-PC:~/OpenLaneUser/designs/fsm$ explorer.exe traffic_fsm.pdf
```

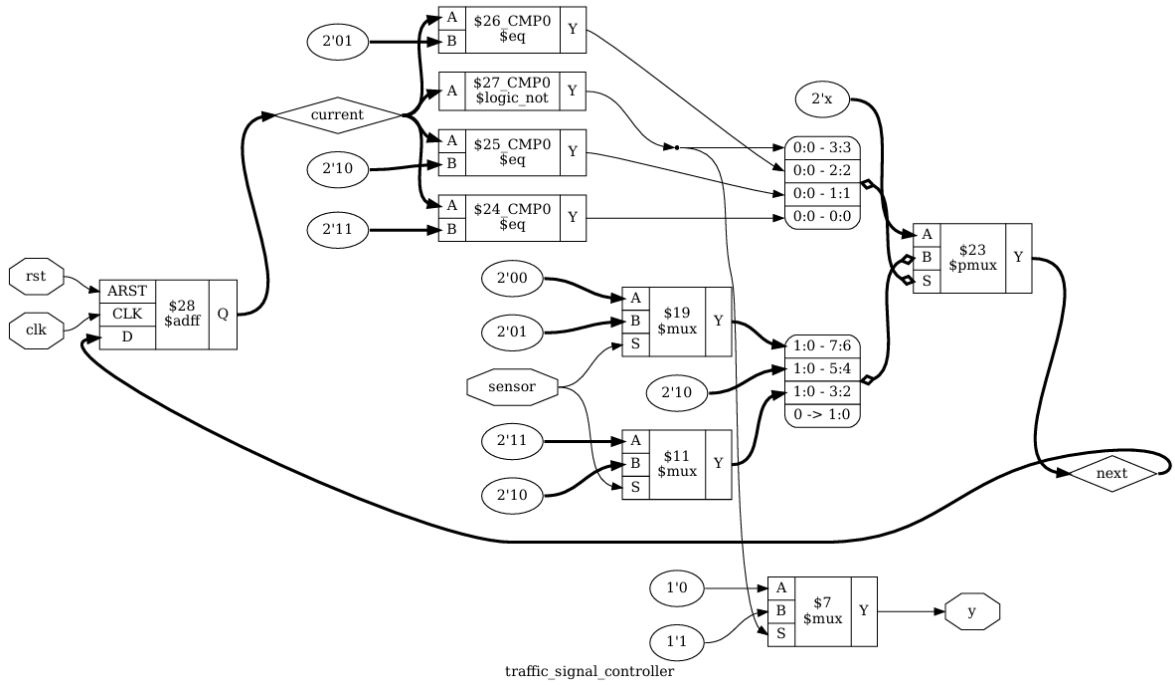


Figure 24.3: RTL schematic of the rising-edge detector generated by Yosys

24.8 Waveform

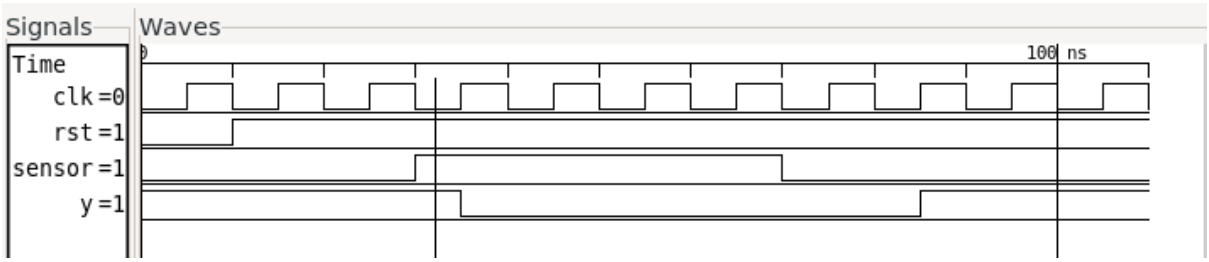


Figure 24.4: GTKWave waveform of the rising-edge detector

24.9 Result

The sensor-based traffic signal controller was successfully designed and verified using Verilog. Functional simulation was performed using Icarus Verilog and the resulting waveform was verified using GTKWave. Yosys synthesis successfully generated the RTL representation of the four-state FSM.

The implementation demonstrates the complete RTL design and verification flow for a simple FSM-based digital system and provides a suitable starting point for further ASIC implementation using OpenLane.

24.10 Conclusion

A four-state sensor-based traffic signal controller was successfully implemented using Verilog. The controller responds to vehicle detection and changes the traffic signal through the required GREEN–YELLOW–RED sequence. The design was functionally verified through simulation and its synthesized RTL structure was examined using Yosys.